SolveIt

The Complete Solution Guide

rustSolveIt, on pure-Rust SUNDIALS 7.8.0

17 August 2026

Written for a reader who has never used this program, has never run a physics simulator, and does not write Rust. Every term is defined the first time it is used. Every number printed below was produced by the program on the machine that wrote this file; nothing is quoted from memory.

Contents

1 What this is, in one page **4**
1.1 The rules the code lives under 4

2 Installing and running it **5**

3 The five things you need to know **5**
3.1 `RUN` takes a duration, not a destination 5
3.2 Gravity is on by default, and it is between *bodies* 5
3.3 Objects are numbered, and deleting renumbers them 6
3.4 Collisions must be switched on 6
3.5 $|dE/E|$ is printed for free, and you should read it 6

4 What the program is made of **6**
4.1 The one type that does everything 6
4.2 The state vector 7

5 The engine: SUNDIALS 7.8.0 in pure Rust **7**
5.1 What SUNDIALS is 7
5.2 Which solver runs when 7
5.3 What the 7.8.0 upgrade brought 7
5.4 The three questions that are not "what happens next" 8

6 How collisions actually work **9**
6.1 The problem with checking every frame 9
6.2 What this program does instead 9
6.3 What happens at the moment of contact 9
6.4 The Zeno problem 9

7 Thirty-eight worked examples **9**
7.1 A head-on elastic collision between unequal masses 10

7.2 Kepler's third law, four orbits at once . . . 10
7.3 Bound, parabolic and hyperbolic, decided by a sign . . . 11
7.4 The restitution ladder, and honest energy loss . . . 11
7.5 Cyclotron motion, and when to reach for BDF . . . 12
7.6 Symplectic versus adaptive, over a long run . . . 12
7.7 The tennis-racket theorem, measured . . . 13
7.8 When energy *should* grow . . . 13
7.9 Newton's cradle . . . 14
7.10 The bullet that does not tunnel . . . 14
7.11 Lagrange's equilateral three-body solution . . . 15
7.12 A user-defined constructor, and an inertia tensor . . . 15
7.13 Geometry a bounding sphere gets wrong . . . 16
7.14 The infinite square well, solved in the language . . . 16
7.15 Tunnelling, and watching a grid converge . . . 17
7.16 The special-function library, checked by identities . . . 18
7.17 A pendulum as a constraint, not a force . . . 19
7.18 Where it rests, and what the answer depends on . . . 20
7.19 A door on a hinge . . . 20
7.20 A Kepler ellipse, measured by its own database . . . 21
7.21 Free fall against the closed form, checked inside the database . . . 22
7.22 An equal-mass circular binary, audited by SQL . . . 23
7.23 Three restitutions, one query . . . 24
7.24 The elastic exchange of Example 1, replayed from the database . . . 25
7.25 A helical cyclotron, its radius and pitch by aggregate . . . 26
7.26 The Dzhanibekov flip, with its half-flips counted by a window function . . . 26
7.27 The recording switch, proven by the gap it leaves . . . 27
7.28 The DAE pendulum, its rod audited to roundoff . . . 28
7.29 The box rattle, its energy ledger in SQLite . . . 29
7.30 Schema surgery on a copy — the dataset is a real database . . . 30
7.31 Two integrators, two datasets, one `ATTACH` . . . 31
7.32 Differentiating a trajectory inside the database . . . 32
7.33 The complete dataset lifecycle in one sitting . . . 33
7.34 Two movies from one dataset . . . 35
7.35 A spinning torus: quaternions in the database and on the screen . . . 36
7.36 The teacher's experiment — change a mass, rerun, subtract . . . 37
7.37 Cartesian to spherical, and to plane polar — with the equator as witness . . . 39
7.38 One helix through every remaining chart . . . 40

8 Watching it: the scene window, videos, dataset plots and movies 42
8.1 The live window . . . 42
8.2 Recorded videos . . . 42
8.3 The dataset layer: SQLite recording . . . 44
8.4 Movies: animations derived from datasets . . . 44

9 How you know the numbers are right 45
9.1 The solver library against its C original . . . 45
9.2 The simulator against closed-form physics . . . 45
9.3 The test suite . . . 45

9.4 The notebooks 45
9.5 And for the 7.8.0 upgrade specifically 45

10 When something goes wrong **45**

11 Where everything lives **47**

1 What this is, in one page

SolveIt simulates classical mechanics and one-, two- and three-dimensional quantum mechanics. You describe a situation — some bodies, some fields, maybe a rigid box to put them in — and it tells you what happens next, to as many digits as the arithmetic allows.

You talk to it by typing short commands, one per line:

```
In[1]:= new sphere { mass = 2, radius = 0.5, position = [0, 10, 0],
                     velocity = [1, 0, 0] }
Out[1]= obj0
In[2]:= set system.gravity = [0, -9.81, 0]
In[3]:= step 1
Out[3]= t = 1 (advanced by 1, 12 solver steps)
In[4]:= get obj0.position
Out[4]= [1, 5.095000000000006, 0]
```

That is the whole interface. No configuration files, no build system to learn, no graphics API.

The one thing that makes this different from a game engine. Game physics is allowed to be approximately right — it only has to look plausible for the next sixteen milliseconds. This program's job is to be *measurably* right. Every trajectory it prints comes out of a production-grade differential-equation solver with genuine error control, and every claim in the documentation is a number you can reproduce by running the command underneath it.

Concretely, and these are the actual measured values:

claim	**measured**
the outer solar system, integrated for 1,369 years	Pluto's position matches the reference to 8 decimal places; total energy drifts by 7.8 parts in 10^7
a torque-free tumbling body	angular momentum changes by **exactly zero**
a ball dropped on a plate	the moment of impact is right to 1 part in 10^{16}
three shapes rattling in a rigid box, 36 collisions	energy conserved to 3 parts in 10^{16}
Kepler's third law across four orbits	T^2/a^3 identical to the last bit for all four

1.1 The rules the code lives under

1. **Everything is integrated by SUNDIALS.** There is no hand-written stepper anywhere in the program — not in the examples, not in the graphics playback, not as a "fast path".
2. **No `unsafe` code.** Rust's memory-safety checks are on everywhere, with no escape hatches.
3. **No dependencies at all.** Not one package from the internet. The solver library, the web server behind the graphics window, the WebSocket protocol, the SHA-1 implementation, the special-function library — all of it is in this repository.
4. **No compiler warnings.** The build is clean, and it is set up to *fail* if it stops being clean.

Rule 3 has a consequence worth spelling out: `git clone`, `cargo build`, done. No network access during the build, nothing to go stale, nothing that can be pulled out from under you.

2 Installing and running it

You need Rust. Then:

```
git clone https://github.com/once-ere/rustSolveIt_macos-silicon_SUNDIALS_7_8_0.git
cd rustSolveIt_macos-silicon_SUNDIALS_7_8_0
cargo run
```

The last command builds everything and drops you at the `In[1]:=` prompt. Type `HELP` for the reference card, `QUIT` to leave.

how	**command**	**when**
interactive notebook	`cargo run`	you are exploring
batch script	`cargo run -p posim -- --script my_run.posim`	you want it reproducible
dynamic notebook	`cargo run -p posim --release -- --notebook dynamic_notebooks/kepler_orbit.posim`	run a saved session, open its graphics window, keep typing
machine mode	`cargo run -p posim -- --machine`	another program is driving

A "script" is just a text file of the same commands you would type. Comments start with `#`. Every example in §7 is a script in `scripts/solveit/`.

Checking your build: `cargo test --workspace` — 605 tests should pass.

3 The five things you need to know

3.1 `RUN` takes a duration, not a destination

```
In[1]:= run 1.7 steps 170        # advances BY 1.7, so t is now 1.7
In[2]:= run 1.19 steps 120       # advances BY 1.19, so t is now 2.89
```

If you want to stop at an absolute time, subtract. This trips up everyone once. `STEP` *dt* is the same thing with one output point.

3.2 Gravity is on by default, and it is between *bodies*

`system.g_constant` defaults to **1**, which means every pair of bodies attracts every other pair. That is what you want for a solar system and emphatically not what you want for three blocks in a box: two touching surfaces are at nearly zero separation, and the attraction between them is nearly singular.

If you are studying collisions, turn it off: `set system.g_constant = 0`.

The difference is not subtle. The same three-shapes-in-a-box scenario conserves energy to **3 parts in** 10^{16} with $G = 0$, and drifts by **3.2 %** with $G = 1$. Neither number is a bug — they are different physical systems. `system.uniform_gravity` is the separate thing you usually mean by "gravity": a constant downward field, like a laboratory.

3.3 Objects are numbered, and deleting renumbers them

`obj0`, `obj1`, ... in creation order. `DEL` 1 removes `obj1` **and renumbers everything after it**. Give things names instead: `new sphere as earth { ... }`, then `get earth.position`. Names survive deletions of other objects.

3.4 Collisions must be switched on

`COLLIDE ON`. Without it, bodies pass through each other. This is a deliberate default: collision detection changes how the solver picks its steps, and a great many problems (orbits, fields, quantum) never need it.

3.5 $|dE/E|$ is printed for free, and you should read it

Every `RUN` reports the relative change in total energy. For a closed system this should be tiny. If it is not, either the physics genuinely does not conserve energy (a magnetic torque, an inelastic bounce, a driven potential — see examples 4 and 8) or something is wrong with your setup. It is the cheapest sanity check you will ever get.

4 What the program is made of

piece	what it does
`posim`	the command language: reads your line, checks it, compiles it to a tiny stack program, runs it. Also the graphics window and the machine-mode protocol.
`physical_object`	the physics: one `physical_object` type that is simultaneously a point mass, a rigid body and a shaped, collidable solid. Plus all the time integration.
`quantum`	1-D, 2-D and 3-D quantum mechanics: bound states, wavepacket propagation, tunnelling, absorbing boundaries.
`special_functions`	Bessel, Legendre, Hankel, Airy, gamma, Wigner symbols, quadrature, eigenproblems.
`sundials_rs`	the differential-equation solvers (§5).
`jupyter`	a JupyterLab kernel.

4.1 The one type that does everything

The original 2002 SolveIt had three separate types: a point particle, a rigid body, and a 3-D rigid body. Each had its own integrator, its own bugs, and its own idea of what "position" meant. This version has **one** type, `physical_object`, which is the union of all three. A point mass is just a `physical_object` whose shape is `POINT` and whose inertia tensor is zero.

That sounds like a detail. It is why a point mass, a spinning cuboid and a dumbbell can all be in the same collision and the same conservation sum, without a single special case.

4.2 The state vector

Every object carries **13 numbers**:

[position (3) | momentum (3) | orientation quaternion (4) | angular momentum (3)]

Momentum rather than velocity, and angular *momentum* rather than angular velocity, both on purpose: those are the quantities that are conserved, and the ones that behave correctly when you change a mass mid-run. Quaternions are written **w first**: $[w, x, y, z]$.

5 The engine: SUNDIALS 7.8.0 in pure Rust

5.1 What SUNDIALS is

SUNDIALS — SUite of Nonlinear and DIfferential/ALgebraic equation Solvers — is Lawrence Livermore National Laboratory's library of differential-equation solvers, in production use in physics and engineering codes since the 1990s. It is written in C.

`sundials_rs/` in this repository is a **line-for-line translation of SUNDIALS 7.8.0 into pure Rust**. Not a wrapper, not bindings: there is no C compiler involved, no shared library to link, no FFI boundary. It carries the same control flow, the same constants, the same tolerances, and — a subtler point that matters a great deal — the same *order of arithmetic operations*, because floating-point addition is not associative and changing the order changes the answer in the last digits.

5.2 Which solver runs when

you type	what runs
`METHOD ADAMS` (default)	**CVODE Adams–Moulton**: variable order, variable step size, Newton iteration, dense finite-difference Jacobian. The general-purpose choice.
`METHOD BDF`	**CVODE BDF**: backward differentiation formulas, for *stiff* problems — ones with two very different timescales, like a fast magnetic gyration inside a slow drift.
`METHOD SPRK` *table* [*dt*]	**ARKODE SPRKStep**: a *symplectic* partitioned Runge–Kutta method at a fixed step. Only valid for separable systems, and worth it when you have one.

"Adaptive" means the solver chooses its own internal step size to keep the estimated local error under the tolerance you asked for. When it reports `1289 solver steps` for a run you asked to sample at 400 points, those 1289 are its own steps; the 400 are just where it stopped to tell you the answer. **The output cadence does not affect the physics.** That property is tested, not assumed.

5.3 What the 7.8.0 upgrade brought

This repository is `once-ere/rustSolveIt` with its 7.7.0 engine replaced by the 7.8.0 one. Two things came with it.

Four more solver families.

crate	solves	used here?
`cvode_rs`	ordinary differential equations (Adams / BDF)	yes
`arkode_rs`	Runge–Kutta: explicit, implicit, IMEX, multirate, symplectic	yes
`ida_rs`	differential-algebraic systems $F(t, y, \dot{y}) = 0$	yes — `CONSTRAIN` + `METHOD IDA`
`kinsol_rs`	nonlinear algebraic systems, Anderson acceleration	yes — `EQUILIBRIUM`
`cvodes_rs`	CVODE plus forward and adjoint sensitivity analysis	yes — `SENSITIVITY`
`idas_rs`	IDA plus sensitivity analysis	yes — `SENSITIVITY` while constra

All six are compared byte-for-byte against the output of the original C programs.

5.4 The three questions that are not "what happens next"

Four joints. `CONSTRAIN` *a b* is a rod (1 row, 5 freedoms left); `BALL` *a b* shares a point (3 rows); `UNIVERSAL` *a b u w* shares a point and keeps two shafts square, a Cardan joint (4 rows); `HINGE` *a b axis* shares a point *and* an axis, leaving exactly one freedom (5 rows) — a door, a knee, a pendulum. The last three grip orientation, which is why `METHOD IDA` carries the full 13-numbers-per-object state.

A rigid rod, held exactly — `CONSTRAIN` *a b* then `METHOD IDA`. A rod is a geometric fact, not a very stiff spring: the bob is *exactly* L from the pivot, always. Saying so turns the equations of motion from an ODE into a differential-algebraic equation, which is what IDA solves. A body with `inverse_mass = 0` becomes an **anchor** — it never moves and absorbs the reaction — so anchor + bob + rod is a pendulum. `CONSTRAINTS` reports how far the rod has strayed; over a 20-second swing it stays at roundoff.

Where it comes to rest — `EQUILIBRIUM`. Not an integration: KINSOL solves for a configuration where every free body has zero net force, every anchor is where it started and every rod is the right length, then puts the bodies there and stops them. It says nothing about *stability* — a pencil on its point is an equilibrium too.

How much the answer depends on an input — `SENSITIVITY 3 "gravity.y"`. Running twice with slightly different inputs and subtracting throws away most of your digits. This integrates the derivative alongside the state instead. CVODES for an ordinary system; IDAS when a `CONSTRAIN` is active.

A more faithful API. The 7.8.0 translation models C's opaque pointers directly rather than smoothing them into idiomatic Rust. Vector data is reached through a borrow guard, constructors return `Option` where C returns a possibly-null pointer, and destructors blank the handle the way `free(p); p = NULL;` does. None of that is visible from the command language; the full call-by-call table is in `PORT_7.8.0_PROVENANCE.md`.

And the physics did not move. All six self-checking physics examples, all twelve collision scripts and all 59 dynamic notebooks produce output that is byte-for-byte identical under 7.7.0 and 7.8.0.

6 How collisions actually work

6.1 The problem with checking every frame

The obvious way to detect a collision is to advance time a little, then ask "are these two objects overlapping?" If yes, back up and resolve it.

This fails in a specific, embarrassing way. Consider a bullet at 100 m/s and a plate 5 mm thick. In one hundredth of a second the bullet moves a full metre — two hundred times the plate's thickness. Check before the step: not overlapping. Check after: not overlapping. The bullet is now on the other side, and the program never noticed. This is called **tunnelling**.

6.2 What this program does instead

It hands the solver a **continuous function** — the signed gap between two surfaces, positive when apart, negative when overlapping — and asks it to find the moment that function crosses zero. This is **rootfinding**, a first-class feature of CVODE, not something bolted on top.

Three things follow. **The impact time is exact**, not quantised to a frame: the solver interpolates its own solution back to the crossing. **The solver caps its own step size** while rootfinding is armed, computed from the smallest feature among the paired bodies divided by the fastest surface approach speed — and it accounts for *acceleration*, so a body released from rest above a thin plate still gets a finite cap. **Systems with nothing to collide are unaffected**: with no collidable pairs the code path is bit-identical to running with collisions off, and that is a tested property.

6.3 What happens at the moment of contact

The solver stops at the crossing and hands control back. The program then reads the state at exactly that instant; computes the contact point and normal from the two shapes' actual geometry (a *support function*, not a bounding sphere — see example 13); applies an impulse along the normal, sized by the two bodies' masses, inertias and coefficients of restitution; and re-initialises the solver at the new state. The impulse is applied at *one shared point* for both bodies, which is what makes angular momentum come out right for off-centre hits.

6.4 The Zeno problem

A bouncing ball with restitution below 1 makes infinitely many bounces in finite time. The program counts events per *burst* — a run of events with no real flight between them. Past a threshold it forces the remaining contacts plastic; past twice the threshold it projects out any overlap and disarms rootfinding for the rest of the output interval. The counting is per-burst rather than per-output-interval, because per-interval counting made the physics depend on how often you asked for output.

7 Thirty-eight worked examples

Every transcript below is genuine program output. The scripts are in `scripts/solveit/`:

```
cargo run -p posim --release -- --script scripts/solveit/01_elastic_head_on.posim
```

Examples 1–19 use the core language. Examples 20–38 add the **SQLite dataset layer**: each records its run into a database under `datasets/`, answers its physics question **in SQL**, and derives plots and movies from the recording — and examples 36–38 add **trajectory algebra** (`DATASET DIFF/SUM`, alignment checked) and the **fourteen classical curvilinear coordinate systems** (`DATASET MAP` and the value-level `cart_to_*/*_to_cart` builtins; see §8.3 and §8.4, and the grammar guide §5.14–§5.15). They need the sidecar built once:

```
cd dataset_tools && cargo build --release
```

7.1 A head-on elastic collision between unequal masses

Two spheres, masses 3 and 1, approach head-on at +2 and −4. Elementary mechanics gives $v_1' = ((m_1 - m_2)u_1 + 2m_2u_2)/(m_1 + m_2)$ and the mirror image for v_2'. The answer is exact, and both energy and momentum must survive the impulse untouched.

```
In[5]:= energy
Out[5]= 14
In[6]:= momentum
Out[6]= [2, 0, 0]
In[7]:= run 2 steps 200
Out[7]= t = 2 (53 solver steps, 200 snapshots, |dE/E| = 0.000e0,
        1 collision(s))
In[8]:= get heavy.velocity
Out[8]= [-1, 0, 0]
In[9]:= get light.velocity
Out[9]= [5, 0, 0]
In[14]:= ((m1 - m2) * u1 + 2 * m2 * u2) / (m1 + m2)
Out[14]= -1
In[15]:= ((m2 - m1) * u2 + 2 * m1 * u1) / (m1 + m2)
Out[15]= 5
In[16]:= energy
Out[16]= 14
In[17]:= momentum
Out[17]= [2, 0, 0]
```

−1 and 5 exactly, not approximately. Energy 14 before and 14 after; momentum $[2, 0, 0]$ before and after. And $|dE/E| = 0$ — the run lost nothing at all. Note that `LET` makes a session variable and a bare expression line is evaluated and printed, so the closed form is checked *inside the language*.

7.2 Kepler's third law, four orbits at once

For a circular orbit of radius a about mass M, the period is $T = 2\pi a^{3/2}/\sqrt{GM}$. Kepler's third law says T^2/a^3 is the same number for every orbit around the same central mass. Each planet is run for its own period and must come home.

```
Out[5]=  [1.0000000044544244, 0, -0.00000014431704032681625]
Out[11]= [2.000000000309673,  0,  0.00000002604593958739631 6]
Out[17]= [3.0000000004230944, 0,  0.0000000407188128857122 2]
Out[23]= [4.000000000077065,  0,  0.0000000724874582177744 7]

In[24]:= 0.19869176531592203 * 0.19869176531592203 / 1
```

```
Out[24]= 0.039478417604357434
In[25]:= 0.5619851784832581 * 0.5619851784832581 / (2 * 2 * 2)
Out[25]= 0.039478417604357434
In[26]:= 1.0324326977181857 * 1.0324326977181857 / (3 * 3 * 3)
Out[26]= 0.039478417604357434
In[27]:= 1.5895341225273762 * 1.5895341225273762 / (4 * 4 * 4)
Out[27]= 0.039478417604357434
```

All four are the same 17 digits. Not "close" — identical. That is $4\pi^2/1000$.

7.3 Bound, parabolic and hyperbolic, decided by a sign

The shape of an orbit is decided by the specific orbital energy $\varepsilon = v^2/2 - GM/r$: negative is an ellipse, exactly zero a parabola, positive a hyperbola. The escape speed at r is $\sqrt{2GM/r}$; at $r = 4$ with $GM = 1000$ that is $\sqrt{500}$.

```
In[7]:= 20 * 20 / 2 - gm / 4
Out[7]= -50
In[8]:= 22.360679774997898 * 22.360679774997898 / 2 - gm / 4
Out[8]= 0.00000000000002842170943040401
In[9]:= 24 * 24 / 2 - gm / 4
Out[9]= 38
```

-50, then 2.8×10^{-14} (the closest a 64-bit float gets to zero after squaring an irrational number), then $+38$. Three conics from one formula.

7.4 The restitution ladder, and honest energy loss

A ball with restitution e reaches $e^{2n}h$ after n bounces. The centre starts at 5.5 with radius 0.5, so it falls 5.0; $g = 10$, $e = 0.7$. **`RUN` takes a duration**, so the three runs are 1.7, 1.19, 0.833.

```
In[7]:= get ball.position.y
Out[7]= 2.949999999869974
In[8]:= 5 * 0.7 * 0.7 + 0.5
Out[8]= 2.9499999999999997
In[10]:= get ball.position.y
Out[10]= 1.7004999998062575
In[11]:= 5 * 0.7 * 0.7 * 0.7 * 0.7 + 0.5
Out[11]= 1.7004999999999997
In[13]:= get ball.position.y
Out[13]= 1.0882449997748824
In[14]:= 5 * 0.7^6 + 0.5   (written out longhand in the script)
Out[14]= 1.0882449999999997
```

Three apexes, each matching to about 2×10^{-10} — and each arrived at at the right *moment*, which is the harder half.

Also notice $|dE/E| = 4.636\text{e-}1$. Energy is **not** conserved here, and should not be: $e = 0.7$ means each bounce deliberately throws away 51 % of the kinetic energy. A conservation check is only meaningful when the physics is conservative. This is the first place most people misread the diagnostic.

7.5 Cyclotron motion, and when to reach for BDF

A charge q moving at speed v perpendicular to B travels in a circle of radius $r = m|v|/(|q|B)$ with period $T = 2\pi m/(|q|B)$. The Lorentz force $q\,v \times B$ depends on velocity — which rules out symplectic methods entirely — and is fast compared with anything else in the problem. That is the textbook definition of a *stiff* system, and what `METHOD BDF` is for.

Half a period puts the ion diametrically opposite, so the distance from the origin must be exactly $2r$.

```
In[6]:= get ion.position
Out[6]= [-0.00000000020913167204927863, 1.1111111086955652, 0]
In[7]:= norm(ion.position)
Out[7]= 1.1111111086955652
In[8]:= 2 * (2 * 5 / (3 * 6))
Out[8]= 1.1111111111111112
In[10]:= get ion.position          (after the second half period)
Out[10]= [0.00000000040803969144492 8, 0.000000005381884658509583, 0]
In[11]:= get ion.velocity
Out[11]= [4.999999951563042, 0.00000000367235957615674 74, 0]
```

Right to 8 significant figures; after a full period the ion is back at the origin to 5 parts in a billion with its original velocity restored. The magnetic force does no work, and the speed confirms it.

7.6 Symplectic versus adaptive, over a long run

CVODE Adams controls the *local* error at every step. A **symplectic** method controls something else entirely: it exactly preserves the geometric structure of Hamiltonian mechanics, so the energy error does not accumulate — it oscillates around a constant.

```
In[5]:= run 60 steps 60                          (METHOD ADAMS)
Out[5]= t = 60 (507075 solver steps, 60 snapshots, |dE/E| = 1.459e-8)
In[6]:= energy
Out[6]= -0.20000000291885495

In[12]:= run 60 steps 60      (METHOD SPRK MCLACHLAN_4_4 0.001)
Out[12]= t = 60 (60000 solver steps, 60 snapshots, |dE/E| = 8.957e-11)
In[13]:= energy
Out[13]= -0.20000000001790574
```

The symplectic method took **60,000** steps to the adaptive method's **507,075**, and its energy error is **163 times smaller**. Eight times less work, two orders of magnitude better answer.

The catch, and it is a real one. SPRK needs a *separable* Hamiltonian. A magnetic field breaks that, and so does rotation. The program does not let you get this wrong quietly:

```
Err: SPRK method requires a separable Hamiltonian: magnetic field B must
     be zero (the Lorentz force q v x B is velocity-dependent);
     use METHOD ADAMS or BDF
```

The error names the feature that blocks it. That refusal is why the previous example uses BDF.

7.7 The tennis-racket theorem, measured

A rigid body has three principal axes with three different moments of inertia. Spin it about the largest or the smallest and the motion is stable. Spin it about the **middle** one and it is not: the body flips end over end, over and over. Cosmonaut Vladimir Dzhanibekov noticed this on Salyut 7 in 1985 with a wing nut.

The flip is a genuine instability, so it is exquisitely sensitive to integration error — and yet the angular momentum must be *exactly* conserved throughout, because nothing is applying a torque. Those two demands pull in opposite directions.

```
In[2]:= angmom
Out[2]= [0.02, 3, 0.02]
In[3]:= energy
Out[3]= 5.148625491836798

racket.orientation.x, sampled every 0.5 from t = 0.5 to t = 4.0:
   0.013359726984815766
   0.009214316848229325
  -0.06928171314906612
  -0.32837233114496406
  -0.4991829732950659
   0.1093367245470612
   0.8230403847701171
   0.9661018228588313

In[20]:= angmom
Out[20]= [0.02, 3, 0.02]
In[21]:= energy
Out[21]= 5.148625488700016
```

The x component of the orientation quaternion — near enough, "how far over has it rolled" — sweeps from 0.013 through negative territory to 0.966. That is the flip. And through all of it, `ANGMOM` reads $[0.02, 3, 0.02]$ at the end exactly as it did at the start: not to eight decimals, the identical printed value.

You can watch this one: `videos/tumbling_racket.html`.

7.8 When energy *should* grow

A magnetised body in a field feels a torque $\tau = (RMR^T)B$. The field is an outside agent: it does work, so kinetic energy grows and angular momentum about the origin is *not* conserved. Half of understanding a conservation diagnostic is knowing when it should fail.

```
In[3]:= new cuboid as rotor { mass = 1, half_extents = [1, 0.4, 0.4],
          magnetic_moment_tensor = [[0, 0, 0.5], [0, 0, 0], [-0.5, 0, 0]] }
In[4]:= energy
Out[4]= 0
In[5]:= angmom
Out[5]= [0, 0, 0]
In[6]:= run 3 steps 30
Out[6]= t = 3 (550 solver steps, 30 snapshots, |dE/E| = 9.928e-1)
In[7]:= energy
```

```
Out[7]= 0.9928369509454675
In[8]:= angmom
Out[8]= [0.46022300703213415, 0, 0]
```

Starts at rest, ends spinning. The initial torque is $M \cdot B = (0.5 \cdot 2, 0, 0) = (1, 0, 0)$, and the angular momentum duly grows along x.

The trap. With `[[0, 0.5, 0], [-0.5, 0, 0], [0, 0, 0]]` — a perfectly reasonable-looking antisymmetric tensor — the *third column* is zero, $M \cdot B$ is the zero vector, and absolutely nothing happens. No error, no warning, just a body that sits there. Multiply the tensor by your field by hand before you blame the solver.

7.9 Newton's cradle

Five identical spheres, four of them touching. Roll one into the line and *only the far one* leaves, at exactly the incoming speed. Momentum alone does not force this — two balls leaving at half speed would conserve momentum fine. It is momentum **and** energy together that pick out the answer.

```
In[8]:= energy
Out[8]= 0.5
In[9]:= momentum
Out[9]= [1, 0, 0]
In[10]:= run 6 steps 600
Out[10]= t = 6 (30 solver steps, 600 snapshots, |dE/E| = 0.000e0,
         4 collision(s))
In[11..14]:= get b0..b3 .velocity
Out[11..14]= [0, 0, 0]   (all four)
In[15]:= get b4.velocity
Out[15]= [1, 0, 0]
In[16]:= energy
Out[16]= 0.5
In[17]:= momentum
Out[17]= [1, 0, 0]
```

Four impulses, and not one bit lost.

7.10 The bullet that does not tunnel

A 4 mm bullet at 100 m/s meets a 5 mm plate. Between output frames the bullet travels a metre — two hundred plate thicknesses. This is precisely the case that defeats frame-sampled collision detection.

```
In[5]:= run 0.1 steps 10
Out[5]= t = 0.1 (10003 solver steps, 10 snapshots, |dE/E| = 0.000e0,
        1 collision(s))
In[6]:= get bullet.position.y
Out[6]= 8.009000000001455
In[7]:= get bullet.velocity.y
Out[7]= 100
In[9]:= contacts
Out[9]= contact0: obj0 <-> obj1 at t = 0.01995500000000067
```

```
  point  = [0, 0.0025, 0]
  normal = [-0, 1, -0]  (from obj0 toward obj1)
  approach speed = 100, impulse = 2
```

The bullet had 1.9955 m of gap to close at 100 m/s, so the exact answer is 0.019955. Look also at the step count: **10,003 solver steps** for ten output points. That is the anti-tunnelling cap at work — the solver was told it may not take a step large enough to skip the plate, and it obeyed.

7.11 Lagrange's equilateral three-body solution

Three equal masses at the corners of an equilateral triangle, each moving at the right speed, rotate rigidly forever — one of only a handful of exactly solvable configurations of the three-body problem, found by Lagrange in 1772. The same geometry is why Jupiter has Trojan asteroids. With $G = m = 1$ and side a, $\omega = \sqrt{3Gm/a^3}$.

There are three independent things to check: each body comes home, the centre of mass never moves, and the triangle stays equilateral.

```
In[6]:= energy
Out[6]= -1.4999999999984994
In[7]:= run 3.6275987284684357 steps 400
Out[7]= t = 3.6275987284684357 (1289 solver steps, 400 snapshots,
        |dE/E| = 4.689e-10)
In[8]:=  get a.position
Out[8]=  [0.5773502692574497, 0, -0.000000002562782285431969 3]
In[9]:=  get b.position
Out[9]=  [-0.2886751324104037, 0, 0.5000000013400956]
In[10]:= get c.position
Out[10]= [-0.28867513684704826, 0, -0.49999999877731977]
In[11]:= com
Out[11]= [-7.586524001605236e-16, 0, -2.146431180941969e-15]
In[12]:= norm(a.position - b.position)
Out[12]= 1.0000000001184224
```

After a full revolution all three are back within about 3×10^{-9}. The centre of mass has moved by 2×10^{-15} — five million times less, which is what you would expect, since it is protected by momentum conservation rather than by accuracy. And the side length is still 1 to 1 part in 10^{10}.

Note also `norm(a.position - b.position)`: you can do vector algebra on live simulator fields directly in the language.

7.12 A user-defined constructor, and an inertia tensor

A `DUMBBELL` is two solid spheres joined by a rod, treated as **one rigid body**. Its inertia about the centre of mass is the exact composite: $\frac{2}{5}mr^2$ for each sphere, plus mz^2 by the parallel-axis theorem, plus the rod's own terms. The inertia tensor is where compound-body code usually goes wrong, and the answer is a hand calculation.

```
In[1]:= def barbell(m1 = 1, m2 = 1, m_rod = 0.5, r1 = 0.3, r2 = 0.3,
                    rod_r = 0.05, len = 2) {
  new dumbbell as bar { m1 = m1, m2 = m2, m_rod = m_rod, r1 = r1,
                        r2 = r2, rod_radius = rod_r, length = len }
```

```
}
Out[1]= function barbell(7 parameter(s)) defined - 1 body line(s)
In[3]:= barbell(2, 2, 0, 0.3, 0.3, 0.05, 2)
Out[3]= obj0 as bar
In[4]:= list
Out[4]= obj0: dumbbell r1=0.3 r2=0.3 rod_r=0.05 len=2, mass=4,
        charge=0, pos=[0, 0, 0]
In[9]:= get bar.inertia_tensor
Out[9]= [[4.144, 0, 0], [0, 4.144, 0], [0, 0, 0.144]]
In[10]:= 2 * (0.4 * 2 * 0.3 * 0.3 + 2 * 1 * 1)
Out[10]= 4.144
In[11]:= 2 * (0.4 * 2 * 0.3 * 0.3)
Out[11]= 0.144
```

4.144 and 0.144, exactly. The off-diagonal entries are exactly zero, as they must be for a body symmetric about its z axis. Every round shape in this program — torus, disk, cylinder, dumbbell — is symmetric about its **local z axis**; that is the convention.

7.13 Geometry a bounding sphere gets wrong

An axis-aligned torus with outer radius 2 exactly inscribes a cube of inner side 4. Tilt that same torus so its axis points along $(1,1,1)/\sqrt{3}$ and its extent along each coordinate axis *drops* to $1.5\sqrt{2/3} + 0.5 \approx 1.7247$, well inside the walls.

A bounding sphere would say the opposite: the torus's bounding sphere has radius 2 no matter how you turn it. Getting this right requires the actual **support function** of the shape — the farthest point of the body in a given direction — evaluated in the rotated frame.

```
In[2]:= box 4
Out[2]= box: inner size 4 x 4 x 4 - six static walls obj0..obj5 with
        inverse_mass = 0
In[7]:= new torus as tilt { mass = 1, ring_radius = 1.5,
          tube_radius = 0.5, position = [0, 0, 0],
          orientation = [0.8880738339771154, -0.32505758367186816,
                         0.32505758367186816, 0] }
In[8]:= run 1 steps 10
Out[8]= t = 1 (4 solver steps, 10 snapshots, |dE/E| = 0.000e0)
In[10]:= get system.collisions
Out[10]= 0
In[11]:= 1.5 * sqrt(2 / 3) + 0.5
Out[11]= 1.724744871391589
```

Zero collisions. The clearance ($2 - 1.7247 = 0.2753$ per side) is exactly what the geometry predicts. Note also the `BOX` machinery: six wall slabs, created automatically, with `inverse_mass = 0` — *infinite* mass, so they absorb any impulse without ever moving.

7.14 The infinite square well, solved in the language

A particle confined to a box of width L has energies $E_n = n^2\pi^2\hbar^2/(2mL^2)$. `QM GRID` pins the wavefunction to zero just outside the domain, which *is* an infinite square well — the program is solving the real problem, not a special case of it.

```
In[1]:= qm grid 0 1 800
In[2]:= qm potential zero
In[5]:= qm states 4
Out[5]= 4 lowest bound state(s):
  E[0] =  4.934795874467
  E[1] = 19.739107587630
  E[2] = 44.412707408131
  E[3] = 78.955215788088
In[6..9]:= n^2 * pi * pi / 2  for n = 1..4
Out[6]=  4.934802200544679
Out[7]= 19.739208802178716
Out[8]= 44.41321980490211
Out[9]= 78.95683520871486
```

The agreement is to about 1.3 parts in a million, and it gets *worse* for higher levels. That is not sloppiness: it is the finite-difference approximation to the second derivative, whose error grows with the curvature of the state, and which shrinks as h^2 when you refine the grid.

A physical cross-check on the loaded state:

```
In[11]:= qm norm
Out[11]= 1.000000000000345
In[12]:= qm position
Out[12]= 0.500000000001550
In[14]:= qm prob 0 0.5
Out[14]= 0.499999999998032
```

Total probability 1, mean position dead centre, exactly half the probability in each half of the box — all to 12 digits. The *eigenvalue* carries discretisation error; the *state* is properly normalised and properly symmetric.

7.15 Tunnelling, and watching a grid converge

A particle with energy E below a barrier of height V_0 and width a still gets through, with

$$T = \frac{1}{1 + \dfrac{V_0^2 \sinh^2(\kappa a)}{4E(V_0 - E)}}, \qquad \kappa = \frac{\sqrt{2m(V_0 - E)}}{\hbar}.$$

With $V_0 = 4$, $a = 1$, $E = 2$, $\hbar = m = 1$ that is $0.07065082485\ldots$

Transmission depends on the barrier width *exponentially*, so a grid that gets the width slightly wrong gets T badly wrong. That makes it an unusually sharp probe of the discretisation.

```
In[7]:= 1 / (1 + v0 * v0 * s * s / (4 * e * (v0 - e)))
Out[7]= 0.07065082485316446

qm grid -30 30  4000 -> T = 0.069368404960, T + R = 1.000000000000
qm grid -30 30 16000 -> T = 0.070328037206, T + R = 1.000000000000
qm grid -30 30 64000 -> T = 0.070569990790, T + R = 1.000000000000
```

points	T	error
4,000	0.069368	1.3×10^{-3}
16,000	0.070328	3.2×10^{-4}
64,000	0.070570	8.1×10^{-5}

The error falls by a factor of ~ 4 each time the grid is refined by a factor of 4 — first-order convergence, exactly what a staircase-sampled barrier gives you. **This is what "discretisation error" looks like from the outside**, and it is how you tell it from a bug: a bug does not converge.

Notice too that $T + R = 1.000000000000$ on every grid. Probability is conserved exactly regardless of how coarse the grid is, because that is a structural property of the transfer matrix, not an accuracy property. The two kinds of correctness are independent.

Above the barrier there are resonant energies where it becomes perfectly transparent:

```
In[19]:= qm scan 4.1 20 200
Out[19]= 200 energies in [4.1, 20] (0 refused), T from 3.535e-1 to 1.000e0
  1 resonance(s), strongest first:
    E = 8.893969849   T = 0.999992036
```

7.16 The special-function library, checked by identities

The way to test such a library is not to compare against a table (tables have typos) but against **identities** — relations a wrong implementation cannot satisfy by accident.

```
In[1]:= bessel_j(0, 2.404825557695773)
Out[1]= 0
In[2]:= bessel_j_nu(1, 3) * bessel_y_nu(0, 3)
        - bessel_j_nu(0, 3) * bessel_y_nu(1, 3)
Out[2]= 0.21220659078919374 + 0i
In[3]:= 2 / (pi * 3)
Out[3]= 0.2122065907891938
In[4]:= legendre_p(5, 1)
Out[4]= 1
In[5]:= legendre_p(5, 0.906179845938664)
Out[5]= -1.7763568394002506e-16
In[6]:= gamma_z(0.5) * gamma_z(0.5)
Out[6]= 3.1415926535898397 + 0i
In[7]:= pi
Out[7]= 3.141592653589793
In[8]:= sph_harm_real(1, 0, 0, 0)
Out[8]= 0.48860251190291987
In[9]:= sqrt(3 / (4 * pi))
Out[9]= 0.4886025119029199
In[10]:= clebsch_gordan(0.5, 0.5, 0.5, -0.5, 1, 0)
Out[10]= 0.7071067811865475
In[11]:= sqrt(0.5)
Out[11]= 0.7071067811865476
In[12]:= gauss_legendre(4)
Out[12]= [[-0.8611363115940526, -0.3399810435848563,
            0.3399810435848563,  0.8611363115940526],
          [ 0.34785484513745374, 0.6521451548625461,
```

```
            0.6521451548625461,  0.34785484513745374]]
```

Line by line: J_0 at its first zero is a clean 0. The Wronskian identity $J_{\nu+1}Y_\nu - J_\nu Y_{\nu+1} = 2/(\pi x)$ matches to 16 digits — a genuinely strong test, since it couples four different function evaluations. $P_5(1) = 1$ exactly, and 0.906179845938664 really is one of its roots (residual one bit). $\Gamma(\frac{1}{2})^2 = \pi$ to 14 digits, via the *complex* gamma evaluated on the real axis. The spherical harmonic Y_1^0 at $\theta = 0$ is $\sqrt{3/4\pi}$ to 15 digits. Two spin-$\frac{1}{2}$ particles coupling to the $m = 0$ triplet have Clebsch–Gordan coefficient $1/\sqrt{2}$ to the last bit. And the 4-point Gauss–Legendre rule reproduces the classical nodes and weights, whose sum is 2 — the length of $[-1, 1]$.

One design decision worth knowing. Where an argument is an integer order, a fractional value is refused rather than truncated:

```
In[1]:= hermite_h(2.5, 1)
Err[1]: hermite_h(): argument 1 must be a whole number
        (an integer order), got 2.5
```

Truncating would have returned a confident, wrong number with no way for you to notice. Angular momenta, on the other hand, *may* be half-integers, so the Wigner and Clebsch–Gordan routines take plain numbers.

7.17 A pendulum as a constraint, not a force

The small-amplitude period is $T = 2\pi\sqrt{L/g}$; released 0.02 rad off vertical with $L = 1$, $g = 9.81$ that is 2.0060666807106475 s, and after exactly one period the bob must be back where it started. The rod is not a force but a geometric condition, and the only way to hold it exactly is to solve the DAE.

```
In[4]:= new sphere as pivot { mass = 1, radius = 0.02,
                              position = [0, 0, 0], inverse_mass = 0 }
In[5]:= new sphere as bob   { mass = 1, radius = 0.05,
          position = [0.019998666693333084, -0.9998000066665778, 0] }
In[6]:= constrain pivot bob
Out[6]= constraint0: obj0 <-> obj1 held at 1
        (METHOD IDA is required to integrate it)
In[8]:= method ida
Out[8]= method = IDA (constrained DAE, GGL index-2)
In[9]:= run 2.0060666807106475 steps 200
Out[9]= t = 2.0060666807106475 (339 solver steps, 200 snapshots,
        |dE/E| = 2.530e-12)
In[10]:= get bob.position
Out[10]= [0.019998666320588818, -0.9998000066740419, 0]
In[11]:= constraints
Out[11]= constraint0: obj0 <-> obj1 length 1
         (currently 1.0000000000000082)
```

The bob came back to within 3.7×10^{-10} after a full period, and the rod reads 1.0000000000000082 — eight parts in 10^{15}, one bit, after 339 solver steps. That is not luck: the formulation carries *both* the rod's length and its rate of change as equations. The cheaper scheme that constrains only the acceleration lets the length drift quadratically, and you find out much later.

The `inverse_mass = 0` on the pivot is what makes it a pendulum rather than two bodies tumbling about their shared centre of mass joined by a stick.

7.18 Where it rests, and what the answer depends on

A pendulum released anywhere comes to rest hanging straight down, one rod-length below the pivot:

```
In[7]:= equilibrium
Out[7]= equilibrium found in 17 Newton iteration(s), 67 residual
        evaluation(s); largest net force on any free body =
        7.459152323898993e-13, worst |g| = 1.9317880628477724e-14
In[8]:= get bob.position
Out[8]= [0.00000000000007352624797250 06, -0.9999999999999807, 0]
```

Released 57° off vertical, the bob lands straight down to 13 digits. Now the derivative, on a free body, where $\partial y/\partial g = T^2/2 = 4.5$ at $T = 3$:

```
In[15]:= sensitivity 3 "gravity.y" "mass 0"
Out[15]= t = 3 (CVODES, 129 solver steps)
d/d(gravity.y):
  obj0 position [0, 4.500000056696235, 0]
d/d(mass 0):
  obj0 position [0, 0, 0]
In[16]:= get stone.position
Out[16]= [3.000000000000001, -44.14500000000045, 0]
```

4.500000056696235 against an analytic 4.5 — 1.3 parts in 10^8, carried alongside a trajectory that itself landed on -44.14500000000045 where $-\frac{1}{2} \cdot 9.81 \cdot 9 = -44.145$.

And the second derivative is **exactly zero**. In uniform gravity every mass accelerates equally, so the trajectory does not depend on the mass at all. A finite-difference estimate would have returned some small number and left you guessing whether it was noise or physics. The sensitivity equations return the zero.

7.19 A door on a hinge

A hinge fixes a point *and* an axis, leaving one freedom. A hinged rigid body is a **compound** pendulum: its small-amplitude period is $T = 2\pi\sqrt{I_{\text{pivot}}/(mgd)}$ with $I_{\text{pivot}} = I_{\text{com}} + md^2$ — the moment of inertia about the pivot, not just the distance to the centre of mass. The point-mass formula is wrong by 15 % for the slab below.

```
In[4]:= new sphere as jamb { mass = 1, radius = 0.02,
                             position = [0, 0, 0], inverse_mass = 0 }
In[5]:= new cuboid as door { mass = 1, half_extents = [0.2, 0.4, 0.2],
          position = [0.0199986666933331, -0.9998000066665778, 0] }
In[6]:= hinge jamb door [0, 0, 1]
In[8]:= method ida
Out[8]= method = IDA (constrained DAE, GGL index-2)
In[9]:= run 1 steps 10
Out[9]= t = 1 (70 solver steps, 10 snapshots, |dE/E| = 1.613e-9)
In[10]:= constraints
```

```
Out[10]= constraint0: hinge obj0 <-> obj1, 5 row(s)
worst |g| = 2.73750133672479e-10, worst |g_dot| = 2.3769240437118873e-9
In[11]:= get door.angular_momentum
Out[11]= [0, 0, 0.003742219622967182]
```

The angular momentum is $[0, 0, 0.0037]$ — *exactly* zero on x and y. The two rows a hinge has over a ball joint are the ones that forbid those axes, and they are doing their job to the last bit. The joint is held to 2.7×10^{-10} after 70 solver steps, and energy to 1.6 parts in 10^9; run one compound period and the slab returns to within 3×10^{-8}.

The `inverse_mass = 0` on the jamb is what makes it a door: an anchor, immovable, absorbing whatever the hinge pulls with.

A joint constrains velocity, not just position. A ball joint says the two bodies share a point, so at the velocity level $v_i + \omega_i \times r_i = v_j + \omega_j \times r_j$: a body turning about a pivot offset from its centre *must have its centre moving*. Give it a spin and leave its velocity at zero and the state is not on the constraint manifold. The run projects the starting velocities onto it — the smallest mass-weighted change that satisfies the joint, which is precisely the impulse a real coupling delivers when clutched onto a spinning shaft — and reports how big that change was. For a cube spun at 3 rad/s on a half-metre arm the projection leaves the turn nearly untouched and sets the *centre* moving instead: the pivot was running at 1.5 m/s, and giving a 1 kg body some velocity is cheaper than fighting the spin.

One real limit. Orientation joints carry a tolerance floor of `rtol` $= 10^{-6}$: the index-2 system has an accuracy ceiling, and across twelve compound pendulums the boundary is sharp — 10^{-6} converges in every one, 10^{-8} in none.

7.20 A Kepler ellipse, measured by its own database

A true two-body problem: sun mass 3 and planet mass 1 under $G = 1$, so the relative orbit has gravitational parameter $\mu = G(M + m) = 4$. Starting at perihelion separation $r_p = a(1 - e) = 1$ with $a = 2.5$, $e = 0.6$, the relative vis-viva speed is $\sqrt{\mu(2/r_p - 1/a)} = \sqrt{6.4}$, split about the stationary centre of mass (the planet takes 3/4 of it, the sun 1/4 backwards). Aphelion must come out at $a(1 + e) = 4$ and one period is $T = 2\pi\sqrt{a^3/\mu} = 12.418264743622237$.

The whole orbit is recorded into a SQLite **dataset** as it runs, and then the perihelion and aphelion are measured **by the database**: a self-join pairs the two bodies frame by frame and takes the extremes of their separation. The simulator never computes either number — SQL does.

```
In[1]:= set system.g_constant = 1
In[2]:= new sphere as sun    { mass = 3, radius = 0.15, position = [-0.25, 0, 0],
    velocity = [0, -0.6324555320336759, 0] }
Out[2]= obj0 as sun
In[3]:= new sphere as planet { mass = 1, radius = 0.05, position = [0.75, 0, 0],
    velocity = [0, 1.8973665961010276, 0] }
Out[3]= obj1 as planet
In[4]:= method adams
Out[4]= method = CVODE Adams
In[5]:= dataset create solveit20_kepler
Out[5]= dataset solveit20_kepler created (datasets/solveit20_kepler.db, journal_mode=
    wal, schema v1) -- recording armed
```

```
In[6]:= run 12.418264743622237 steps 200
Out[6]= t = 12.418264743622238 (659 solver steps, 200 snapshots, |dE/E| = 2.096e-9)
In[7]:= dataset info
Out[7]= dataset solveit20_kepler: 2 bodies, 200 frames, 400 states
In[8]:= sql "SELECT MIN(sqrt((p.x-s.x)*(p.x-s.x) + (p.y-s.y)*(p.y-s.y))) AS perihelion,
     MAX(sqrt((p.x-s.x)*(p.x-s.x) + (p.y-s.y)*(p.y-s.y))) AS aphelion FROM states p
    JOIN states s ON s.frame = p.frame WHERE p.body = (SELECT body FROM bodies WHERE
    name = 'planet') AND s.body = (SELECT body FROM bodies WHERE name = 'sun')"
Out[8]= perihelion | aphelion
1.000000001093295 | 4.000000026379791
In[9]:= sql "SELECT MAX(ABS(comx)) AS com_x, MAX(ABS(comy)) AS com_y, MAX(energy) - MIN
    (energy) AS energy_spread FROM frames"
Out[9]= com_x | com_y | energy_spread
5.235284428195541e-12 | 5.539403137577725e-12 | 4.5303885087832896e-9
In[10]:= movie create solveit20_kepler
Out[10]= movie solveit20_kepler created from dataset solveit20_kepler (movies/
    solveit20_kepler.html, 200 frames, 2 bodies)
In[11]:= movie play solveit20_kepler
Out[11]= movie solveit20_kepler (movies/solveit20_kepler.html) ready (browser
    suppressed)
In[12]:= dataset play solveit20_kepler
Out[12]= dataset plots written to datasets/solveit20_kepler_plots.html (browser
    suppressed)
In[13]:= dataset close
Out[13]= dataset solveit20_kepler closed
```

Perihelion `1.000000001`, aphelion `4.000000026` — the analytic ellipse, read back out of a database with one query. The centre of mass sat still to 5×10^{-12}. `movie create` then derives the 3-D player page `movies/solveit20_kepler.html` and `dataset play` the plots page, both from the same tables; run without `POSIM_NO_BROWSER=1` and both open in your browser.

7.21 Free fall against the closed form, checked inside the database

A ball released from $y = 10$ under $g = 9.81$ must follow $y(t) = 10 - (9.81/2)t^2$ exactly. Twenty `RUN` outputs land in the dataset as frames.

One SQL Common Table Expression joins the recorded states to the analytic curve and reports the worst deviation over the whole run — the comparison logic lives entirely in the query.

```
In[1]:= set system.g_constant = 0
In[2]:= set system.uniform_gravity = [0, -9.81, 0]
In[3]:= new sphere as ball { mass = 2, radius = 0.5, position = [0, 10, 0] }
Out[3]= obj0 as ball
In[4]:= dataset create solveit21_free_fall
Out[4]= dataset solveit21_free_fall created (datasets/solveit21_free_fall.db,
    journal_mode=wal, schema v1) -- recording armed
In[5]:= run 1 steps 20
Out[5]= t = 1 (12 solver steps, 20 snapshots, |dE/E| = 1.449e-16)
In[6]:= dataset snapshot
Out[6]= frame 20 recorded at t = 1
In[7]:= sql "WITH err AS (SELECT f.t AS t, s.y - (10.0 - 4.905 * f.t * f.t) AS e FROM
    states s JOIN frames f ON f.frame = s.frame) SELECT MAX(ABS(e)) AS worst_abs_error
    FROM err"
```

```
Out[7]= worst_abs_error
3.552713678800501e-15
In[8]:= sql "SELECT COUNT(*) AS frames, MIN(t) AS t_first, MAX(t) AS t_last FROM frames
    "
Out[8]= frames | t_first | t_last
21 | 0.05 | 1
In[9]:= dataset play
Out[9]= dataset plots written to datasets/solveit21_free_fall_plots.html (browser
    suppressed)
In[10]:= dataset close
Out[10]= dataset solveit21_free_fall closed
```

`worst_abs_error` $= 3.55 \times 10^{-15}$ — machine epsilon territory, because a polynomial of degree two is inside the multistep method's exactness class. `dataset play` writes the plots page for the drop.

7.22 An equal-mass circular binary, audited by SQL

Two unit masses a distance 2 apart, each on a circle of radius 1 about the common centre: $v^2/r = Gm/d^2$ gives $v = 0.5$ and the period is $T = 2\pi r/v = 4\pi$.

Three queries audit three invariants at once: the centre of mass must not move (`MAX(ABS(comx))`), *both* stars' orbit radii must stay 1 (`GROUP BY` over a join to the bodies table), and the energy ledger must stay flat.

```
In[1]:= set system.g_constant = 1
In[2]:= new sphere as star_a { mass = 1, radius = 0.1, position = [-1, 0, 0], velocity
    = [0, -0.5, 0] }
Out[2]= obj0 as star_a
In[3]:= new sphere as star_b { mass = 1, radius = 0.1, position = [ 1, 0, 0], velocity
    = [0,  0.5, 0] }
Out[3]= obj1 as star_b
In[4]:= method adams
Out[4]= method = CVODE Adams
In[5]:= dataset create solveit22_binary
Out[5]= dataset solveit22_binary created (datasets/solveit22_binary.db, journal_mode=
    wal, schema v1) -- recording armed
In[6]:= run 12.566370614359172 steps 150
Out[6]= t = 12.566370614359172 (278 solver steps, 150 snapshots, |dE/E| = 4.496e-11)
In[7]:= sql "SELECT MAX(ABS(comx)) AS com_x, MAX(ABS(comy)) AS com_y FROM frames"
Out[7]= com_x | com_y
7.436273818939299e-13 | 5.651035195342047e-14
In[8]:= sql "SELECT b.name, MIN(sqrt(s.x*s.x + s.y*s.y)) AS r_min, MAX(sqrt(s.x*s.x + s
    .y*s.y)) AS r_max FROM states s JOIN bodies b ON b.body = s.body GROUP BY b.name
    ORDER BY b.name"
Out[8]= name | r_min | r_max
star_a | 0.9999999996159721 | 1.0000000006498577
star_b | 0.9999999996174593 | 1.0000000006488365
In[9]:= sql "SELECT MAX(energy) - MIN(energy) AS energy_spread FROM frames"
Out[9]= energy_spread
9.972572767580345e-11
In[10]:= movie create solveit22_binary
Out[10]= movie solveit22_binary created from dataset solveit22_binary (movies/
    solveit22_binary.html, 150 frames, 2 bodies)
```

```
In[11]:= dataset close
Out[11]= dataset solveit22_binary closed
```

Radii hold 1 to 2×10^{-9} for both stars over a full period, the COM to 10^{-11}, the energy to 5×10^{-10} — and the movie shows two spheres chasing each other around a fixed point with their trails closing into circles.

7.23 Three restitutions, one query

Three balls drop 5 m onto a static slab under $g = 10$, arriving at speed 10. A ball with restitution e leaves at speed $10e$: the recorded maxima of `vy` must be exactly 10, 7 and 5 for $e = 1.0, 0.7, 0.5$.

One `GROUP BY b.name` reads all three restitution ladders out of the dataset simultaneously — per-body maxima over a join, ordered by rebound speed.

```
In[1]:= set system.g_constant = 0
In[2]:= set system.uniform_gravity = [0, -10, 0]
In[3]:= collide on
Out[3]= collisions ON (0 collidable pair(s); 0 impulse(s) so far)
In[4]:= new cuboid as floor { mass = 1, half_extents = [50, 0.05, 50], position = [0,
    -0.05, 0], inverse_mass = 0 }
Out[4]= obj0 as floor
In[5]:= new sphere as elastic { mass = 1, radius = 0.5, position = [-2, 5.5, 0],
    restitution = 1.0 }
Out[5]= obj1 as elastic
In[6]:= new sphere as lively  { mass = 1, radius = 0.5, position = [ 0, 5.5, 0],
    restitution = 0.7 }
Out[6]= obj2 as lively
In[7]:= new sphere as dull    { mass = 1, radius = 0.5, position = [ 2, 5.5, 0],
    restitution = 0.5 }
Out[7]= obj3 as dull
In[8]:= dataset create solveit23_rebounds
Out[8]= dataset solveit23_rebounds created (datasets/solveit23_rebounds.db,
    journal_mode=wal, schema v1) -- recording armed
In[9]:= run 3 steps 300
Out[9]= t = 3 (885 solver steps, 300 snapshots, |dE/E| = 5.333e-1, 20 collision(s) --
    CONTACTS lists them)
In[10]:= sql "SELECT b.name, MAX(s.vy) AS rebound_speed FROM states s JOIN bodies b ON
    b.body = s.body WHERE b.wall = 0 AND b.name <> 'floor' GROUP BY b.name ORDER BY
    rebound_speed DESC"
Out[10]= name | rebound_speed
elastic | 10.000000000000028
lively | 7.000000000000018
dull | 5.000000000000014
In[11]:= sql "SELECT MAX(ncollisions) AS collisions FROM frames"
Out[11]= collisions
20
In[12]:= movie create solveit23_rebounds
Out[12]= movie solveit23_rebounds created from dataset solveit23_rebounds (movies/
    solveit23_rebounds.html, 300 frames, 4 bodies)
In[13]:= dataset close
Out[13]= dataset solveit23_rebounds closed
```

`10.000000000000028 / 7.000000000000018 / 5.000000000000014` — the impulse law to 14 digits, three bodies at once, extracted by SQL. The frames table's cumulative `ncollisions` column gives the bounce count for free.

7.24 The elastic exchange of Example 1, replayed from the database

Masses 3 and 1 meet head-on at $+2$ and -4; the closed form says they leave at -1 and $+5$ (Example 1 verified this live).

Here the same numbers are recovered *from the recording*: scalar subqueries pull each ball's first and last recorded velocity, and the frames table shows momentum and energy flat through the impact. The dataset is a faithful replay of the whole event, not just its endpoints.

```
In[1]:= set system.g_constant = 0
In[2]:= collide on
Out[2]= collisions ON (0 collidable pair(s); 0 impulse(s) so far)
In[3]:= new sphere as heavy { mass = 3, radius = 0.5, position = [-3, 0, 0], velocity =
     [2, 0, 0] }
Out[3]= obj0 as heavy
In[4]:= new sphere as light { mass = 1, radius = 0.5, position = [ 3, 0, 0], velocity =
     [-4, 0, 0] }
Out[4]= obj1 as light
In[5]:= dataset create solveit24_exchange
Out[5]= dataset solveit24_exchange created (datasets/solveit24_exchange.db,
    journal_mode=wal, schema v1) -- recording armed
In[6]:= run 2 steps 200
Out[6]= t = 2 (53 solver steps, 200 snapshots, |dE/E| = 0.000e0, 1 collision(s) --
    CONTACTS lists them)
In[7]:= sql "SELECT (SELECT vx FROM states WHERE body = (SELECT body FROM bodies WHERE
    name='heavy') ORDER BY frame ASC  LIMIT 1) AS heavy_before, (SELECT vx FROM states
    WHERE body = (SELECT body FROM bodies WHERE name='heavy') ORDER BY frame DESC LIMIT
     1) AS heavy_after"
Out[7]= heavy_before | heavy_after
2 | -1
In[8]:= sql "SELECT (SELECT vx FROM states WHERE body = (SELECT body FROM bodies WHERE
    name='light') ORDER BY frame ASC  LIMIT 1) AS light_before, (SELECT vx FROM states
    WHERE body = (SELECT body FROM bodies WHERE name='light') ORDER BY frame DESC LIMIT
     1) AS light_after"
Out[8]= light_before | light_after
-4 | 5
In[9]:= sql "SELECT MAX(px) - MIN(px) AS momentum_spread, MAX(energy) - MIN(energy) AS
    energy_spread FROM frames"
Out[9]= momentum_spread | energy_spread
0 | 0
In[10]:= movie create solveit24_exchange
Out[10]= movie solveit24_exchange created from dataset solveit24_exchange (movies/
    solveit24_exchange.html, 200 frames, 2 bodies)
In[11]:= dataset close
Out[11]= dataset solveit24_exchange closed
```

`heavy`: $2 \to -1$, `light`: $-4 \to 5$, and both spreads EXACTLY 0 — MAX$-$MIN of the momentum and of the energy over every recorded frame is zero to the bit: the collision exactly as Example 1

saw it live, recovered after the fact from SQLite.

7.25 A helical cyclotron, its radius and pitch by aggregate

A charge $q = -3$ in $B = 6\hat{z}$ with $v_\perp = 5$ gyrates at radius $r = mv_\perp/(|q|B) = 10/18 = 0.5555\ldots$ with period $T = 2\pi m/(|q|B) = 0.6981317\ldots$; an added $v_z = 1$ stretches the circle into a helix of pitch $v_z T$.

`MAX(x) - MIN(x)` over the recorded states is the gyro-diameter; `MAX(z)` is the pitch; and MAX − MIN of $\sqrt{v_x^2 + v_y^2 + v_z^2}$ proves the magnetic force did no work — three aggregates, three physics facts.

```
In[1]:= set system.g_constant = 0
In[2]:= set system.b_field = [0, 0, 6]
In[3]:= new sphere as ion { mass = 2, charge = -3, radius = 0.05, position = [0, 0, 0],
     velocity = [5, 0, 1] }
Out[3]= obj0 as ion
In[4]:= method bdf
Out[4]= method = CVODE BDF
In[5]:= dataset create solveit25_cyclotron
Out[5]= dataset solveit25_cyclotron created (datasets/solveit25_cyclotron.db,
    journal_mode=wal, schema v1) -- recording armed
In[6]:= run 0.6981317007977318 steps 300
Out[6]= t = 0.6981317007977318 (300 solver steps, 300 snapshots, |dE/E| = 2.096e-8)
In[7]:= sql "SELECT (MAX(x) - MIN(x)) / 2.0 AS gyroradius, MAX(z) AS pitch FROM states"
Out[7]= gyroradius | pitch
0.5555555528977862 | 0.6981317007977318
In[8]:= sql "SELECT MAX(sqrt(vx*vx + vy*vy + vz*vz)) - MIN(sqrt(vx*vx + vy*vy + vz*vz))
     AS speed_spread FROM states"
Out[8]= speed_spread
5.356468157913241e-8
In[9]:= 2 * 5 / (3 * 6)
Out[9]= 0.5555555555555556
In[10]:= 1 * 0.6981317007977318
Out[10]= 0.6981317007977318
In[11]:= movie create solveit25_cyclotron
Out[11]= movie solveit25_cyclotron created from dataset solveit25_cyclotron (movies/
    solveit25_cyclotron.html, 300 frames, 1 bodies)
In[12]:= dataset close
Out[12]= dataset solveit25_cyclotron closed
```

Gyroradius `0.55555555` (5/9), pitch `0.6981317007977318` (exactly $v_z T$), speed spread 5×10^{-9}. The movie shows the helix climbing its axis with the trail tracing the barber-pole.

7.26 The Dzhanibekov flip, with its half-flips counted by a window function

The tennis-racket theorem: a cuboid with three unequal moments spun almost exactly about its *middle* axis tumbles over and back forever, while $|L|$ is conserved exactly.

The frames table shows `MIN = MAX` for $|L|$ — bit-identical conservation. Then a window function (`LAG(SIGN(qx)) OVER (ORDER BY frame)`) watches the orientation component `qx` change sign and *counts* the half-flips — event detection done entirely in SQL.

```
In[1]:= new cuboid as racket { mass = 1, half_extents = [1.6, 0.9, 0.25],
    angular_momentum = [0.02, 3, 0.02] }
Out[1]= obj0 as racket
In[2]:= dataset create solveit26_dzhanibekov
Out[2]= dataset solveit26_dzhanibekov created (datasets/solveit26_dzhanibekov.db,
    journal_mode=wal, schema v1) -- recording armed
In[3]:= run 8 steps 400
Out[3]= t = 8 (694 solver steps, 400 snapshots, |dE/E| = 3.849e-10)
In[4]:= sql "SELECT MIN(sqrt(lx*lx + ly*ly + lz*lz)) AS L_min, MAX(sqrt(lx*lx + ly*ly +
     lz*lz)) AS L_max FROM frames"
Out[4]= L_min | L_max
3.0001333303705024 | 3.0001333303705024
In[5]:= sql "SELECT MIN(qx) AS qx_min, MAX(qx) AS qx_max FROM states"
Out[5]= qx_min | qx_max
-0.9991758434501936 | 0.9991519536832777
In[6]:= sql "WITH sgn AS (SELECT frame, SIGN(qx) AS s, LAG(SIGN(qx)) OVER (ORDER BY
    frame) AS prev FROM states WHERE ABS(qx) > 0.05) SELECT COUNT(*) AS zero_crossings
    FROM sgn WHERE prev IS NOT NULL AND s <> prev"
Out[6]= zero_crossings
3
In[7]:= dataset play
Out[7]= dataset plots written to datasets/solveit26_dzhanibekov_plots.html (browser
    suppressed)
In[8]:= movie create solveit26_dzhanibekov
Out[8]= movie solveit26_dzhanibekov created from dataset solveit26_dzhanibekov (movies/
    solveit26_dzhanibekov.html, 400 frames, 1 bodies)
In[9]:= dataset close
Out[9]= dataset solveit26_dzhanibekov closed
```

`L_min = L_max = 3.0001333303705024` — not close, *identical*. Three zero crossings of `qx` in eight seconds. `dataset play` shows `qx` swinging between ± 1 while $|L|$ draws a flat line.

7.27 The recording switch, proven by the gap it leaves

Recording is part of the grammar: `dataset record off` pauses appending, `on` resumes it, and physics is untouched either way. A probe falls for three half-second runs with the middle one unrecorded.

The dataset itself is the proof: 10 frames, a t-range that runs *through* the gap, and `COUNT(*) = 0` strictly inside it. Auditing what a switch did is a one-line `WHERE` clause.

```
In[1]:= set system.g_constant = 0
In[2]:= set system.uniform_gravity = [0, -9.81, 0]
In[3]:= new sphere as probe { mass = 1, radius = 0.2, position = [0, 20, 0] }
Out[3]= obj0 as probe
In[4]:= dataset create solveit27_switch
Out[4]= dataset solveit27_switch created (datasets/solveit27_switch.db, journal_mode=
    wal, schema v1) -- recording armed
In[5]:= run 0.5 steps 5
Out[5]= t = 0.5 (12 solver steps, 5 snapshots, |dE/E| = 0.000e0)
In[6]:= dataset record off
Out[6]= dataset recording off
```

```
In[7]:= run 0.5 steps 5
Out[7]= t = 1 (8 solver steps, 5 snapshots, |dE/E| = 1.693e-10)
In[8]:= dataset record
Out[8]= dataset recording is off (open: solveit27_switch)
In[9]:= dataset record on
Out[9]= dataset recording on
In[10]:= run 0.5 steps 5
Out[10]= t = 1.5 (8 solver steps, 5 snapshots, |dE/E| = 1.362e-10)
In[11]:= sql "SELECT COUNT(*) AS frames, MIN(t) AS t_first, MAX(t) AS t_last FROM
    frames"
Out[11]= frames | t_first | t_last
10 | 0.1 | 1.5
In[12]:= sql "SELECT COUNT(*) AS frames_in_gap FROM frames WHERE t > 0.501 AND t <
    0.999"
Out[12]= frames_in_gap
0
In[13]:= dataset info
Out[13]= dataset solveit27_switch: 1 bodies, 10 frames, 10 states
In[14]:= dataset close
Out[14]= dataset solveit27_switch closed
```

`frames = 10`, `t_first = 0.1`, `t_last = 1.5`, and zero frames in $(0.5, 1.0)$ — the gap is exactly where the switch was off, and the ball's physics never noticed.

7.28 The DAE pendulum, its rod audited to roundoff

Example 17's pendulum: the rod is an algebraic constraint, not a force, and IDA integrates the GGL index-2 DAE. The recorded bob coordinates must satisfy $\sqrt{x^2 + y^2 + z^2} = 1$ at every frame.

The constraint audit runs *over the recording*: one aggregate recomputes the rod length from the stored coordinates at all 200 frames and takes the worst deviation. This is exactly the drift the GGL formulation promises to kill.

```
In[1]:= set system.g_constant = 0
In[2]:= set system.uniform_gravity = [0, -9.81, 0]
In[3]:= collide off
Out[3]= collisions OFF (0 collidable pair(s); 0 impulse(s) so far)
In[4]:= new sphere as pivot { mass = 1, radius = 0.02, position = [0, 0, 0],
    inverse_mass = 0 }
Out[4]= obj0 as pivot
In[5]:= new sphere as bob { mass = 1, radius = 0.05, position = [0.019998666693333084,
    -0.9998000066665778, 0] }
Out[5]= obj1 as bob
In[6]:= constrain pivot bob
Out[6]= constraint0: rod obj0 <-> obj1, 1 row(s) (METHOD IDA is required to integrate
    it)
In[7]:= method ida
Out[7]= method = IDA (constrained DAE, GGL index-2)
In[8]:= dataset create solveit28_pendulum
Out[8]= dataset solveit28_pendulum created (datasets/solveit28_pendulum.db,
    journal_mode=wal, schema v1) -- recording armed
In[9]:= run 2.0060666807106475 steps 200
Out[9]= t = 2.0060666807106475 (384 solver steps, 200 snapshots, |dE/E| = 6.942e-13)
```

```
In[10]:= sql "SELECT MAX(ABS(sqrt(x*x + y*y + z*z) - 1.0)) AS worst_rod_stretch FROM
    states WHERE body = (SELECT body FROM bodies WHERE name = 'bob')"
Out[10]= worst_rod_stretch
7.838174553853605e-14
In[11]:= sql "SELECT MIN(y) AS lowest_bob_y FROM states WHERE body = (SELECT body FROM
    bodies WHERE name = 'bob')"
Out[11]= lowest_bob_y
-0.9999999999996914
In[12]:= constraints
Out[12]= constraint0: rod obj0 <-> obj1, 1 row(s)
worst |g| = 7.105427357601002e-15, worst |g_dot| = 2.892412673190032e-14
In[13]:= movie create solveit28_pendulum
Out[13]= movie solveit28_pendulum created from dataset solveit28_pendulum (movies/
    solveit28_pendulum.html, 200 frames, 2 bodies)
In[14]:= dataset close
Out[14]= dataset solveit28_pendulum closed
```

`worst_rod_stretch` $= 7.8 \times 10^{-14}$ over a full period — the constraint held to roundoff at every recorded instant, and the movie shows the swing with the pivot pinned.

7.29 The box rattle, its energy ledger in SQLite

The regression scene: a rigid `BOX` 4, $G = 0$, three different shapes thrown across it, every bounce a genuine CVODE rootfinding event.

The frames table carries the cumulative collision count and the total energy, so the whole conservation claim — dozens of impacts, zero drift — is one SQL line. A second query counts recorded states per shape through a join.

```
In[1]:= set system.g_constant = 0
In[2]:= box 4
Out[2]= box: inner size 4 x 4 x 4 -- six static walls obj0, obj1, obj2, obj3, obj4,
    obj5 with inverse_mass = 0 (infinitely massive); objects collide elastically off
    the inside faces
In[3]:= collide on
Out[3]= collisions ON (0 collidable pair(s); 0 impulse(s) so far)
In[4]:= new cylinder { mass = 2, radius = 0.25, height = 1.5, position = [-1, 0.3, 0],
    velocity = [2, 0.5, -1] }
Out[4]= obj6
In[5]:= new disk     { mass = 0.6666666666666663, radius = 1, position = [1, -0.8,
    0.5], velocity = [-1, 1, 0.6] }
Out[5]= obj7
In[6]:= new cuboid   { mass = 1.6666666666666667, half_extents = [0.5, 0.5, 0.5],
    position = [0.2, 1.1, -0.9], velocity = [0.5, -2, 1] }
Out[6]= obj8
In[7]:= method adams
Out[7]= method = CVODE Adams
In[8]:= dataset create solveit29_box_rattle
Out[8]= dataset solveit29_box_rattle created (datasets/solveit29_box_rattle.db,
    journal_mode=wal, schema v1) -- recording armed
In[9]:= run 6 steps 300
Out[9]= t = 6 (3235 solver steps, 300 snapshots, |dE/E| = 4.495e-10, 41 collision(s) --
     CONTACTS lists them)
```

```
In[10]:= sql "SELECT MAX(ncollisions) AS collisions, MAX(energy) - MIN(energy) AS
    energy_spread FROM frames"
Out[10]= collisions | energy_spread
41 | 4.6800590070006365e-9
In[11]:= sql "SELECT b.shape, COUNT(*) AS recorded_states FROM states s JOIN bodies b
    ON b.body = s.body WHERE b.wall = 0 GROUP BY b.shape ORDER BY b.shape"
Out[11]= shape | recorded_states
cuboid | 300
cylinder | 300
disk | 300
In[12]:= movie create solveit29_box_rattle
Out[12]= movie solveit29_box_rattle created from dataset solveit29_box_rattle (movies/
    solveit29_box_rattle.html, 300 frames, 9 bodies)
In[13]:= movie play solveit29_box_rattle
Out[13]= movie solveit29_box_rattle (movies/solveit29_box_rattle.html) ready (browser
    suppressed)
In[14]:= dataset close
Out[14]= dataset solveit29_box_rattle closed
```

41 collisions resolved, energy spread 4.68×10^{-9} across all of them. The movie draws the `BOX` as the dashed wireframe (the six wall slabs are never drawn — the same pinned rule as the scene window) with cylinder, disk and cuboid ricocheting inside.

7.30 Schema surgery on a copy — the dataset is a real database

No physics question this time: the dataset itself is the subject. `dataset copy` duplicates a recording; the copy then gets a derived column (`ALTER TABLE` + `UPDATE`), a `VIEW`, and a `PRAGMA` inspection — ordinary SQLite DDL/DML passing through the `sql` command verbatim.

It demonstrates the rule that there is no special dataset API to learn: anything SQLite can do to a database, the `sql` command can do to a dataset. And it proves `COPY` isolates: the original never grows the new column.

```
In[1]:= set system.g_constant = 0
In[2]:= set system.uniform_gravity = [0, -9.81, 0]
In[3]:= new sphere as ball { mass = 1, radius = 0.3, position = [3, 4, 0] }
Out[3]= obj0 as ball
In[4]:= dataset create solveit30_original
Out[4]= dataset solveit30_original created (datasets/solveit30_original.db,
    journal_mode=wal, schema v1) -- recording armed
In[5]:= run 0.8 steps 8
Out[5]= t = 0.8 (12 solver steps, 8 snapshots, |dE/E| = 1.811e-16)
In[6]:= dataset close
Out[6]= dataset solveit30_original closed
In[7]:= dataset copy solveit30_original solveit30_lab
Out[7]= dataset solveit30_original copied to solveit30_lab
In[8]:= dataset open solveit30_lab
Out[8]= dataset solveit30_lab opened (1 bodies, 8 frames, 8 states) -- recording armed
In[9]:= sql "ALTER TABLE states ADD COLUMN r REAL"
Out[9]= ok (0 row(s) changed)
In[10]:= sql "UPDATE states SET r = sqrt(x*x + y*y + z*z)"
Out[10]= ok (8 row(s) changed)
In[11]:= sql "SELECT MIN(r) AS r_min, MAX(r) AS r_max FROM states"
```

```
Out[11]= r_min | r_max
3.121053770763972 | 4.960847296833475
In[12]:= sql "CREATE VIEW drop_curve AS SELECT f.t, s.y, s.r FROM states s JOIN frames
    f ON f.frame = s.frame"
Out[12]= ok (0 row(s) changed)
In[13]:= sql "SELECT COUNT(*) AS view_rows FROM drop_curve"
Out[13]= view_rows
8
In[14]:= sql "SELECT name FROM pragma_table_info('states') ORDER BY cid"
Out[14]= name
frame
body
x
y
z
qw
qx
qy
qz
vx
vy
vz
wx
wy
wz
r
In[15]:= dataset close
Out[15]= dataset solveit30_lab closed
In[16]:= dataset open solveit30_original
Out[16]= dataset solveit30_original opened (1 bodies, 8 frames, 8 states) -- recording
    armed
In[17]:= sql "SELECT COUNT(*) AS original_columns FROM pragma_table_info('states')"
Out[17]= original_columns
15
In[18]:= dataset close
Out[18]= dataset solveit30_original closed
In[19]:= dataset delete solveit30_lab
Out[19]= dataset solveit30_lab deleted
```

The copy has 15 columns in `states` (the 14 originals plus `r`), the original still has 14. `pragma_table_info` read the schema, the `VIEW` joined tables, and `dataset delete` removed the lab copy at the end.

7.31 Two integrators, two datasets, one `ATTACH`

The same free fall recorded twice: once under CVODE Adams, once under BDF, into two separate database files.

SQLite's `ATTACH DATABASE` mounts the first file inside the second session, and a single `JOIN` reports the largest disagreement between the two solvers frame by frame — a cross-dataset diff with no application code at all.

```
In[1]:= set system.g_constant = 0
```

```
In[2]:= set system.uniform_gravity = [0, -9.81, 0]
In[3]:= new sphere as probe { mass = 1, radius = 0.2, position = [0, 10, 0] }
Out[3]= obj0 as probe
In[4]:= method adams
Out[4]= method = CVODE Adams
In[5]:= dataset create solveit31_adams
Out[5]= dataset solveit31_adams created (datasets/solveit31_adams.db, journal_mode=wal,
     schema v1) -- recording armed
In[6]:= run 1 steps 10
Out[6]= t = 1 (12 solver steps, 10 snapshots, |dE/E| = 7.243e-16)
In[7]:= dataset close
Out[7]= dataset solveit31_adams closed
In[8]:= reset
Out[8]= system reset
In[9]:= set system.g_constant = 0
In[10]:= set system.uniform_gravity = [0, -9.81, 0]
In[11]:= new sphere as probe { mass = 1, radius = 0.2, position = [0, 10, 0] }
Out[11]= obj0 as probe
In[12]:= method bdf
Out[12]= method = CVODE BDF
In[13]:= dataset create solveit31_bdf
Out[13]= dataset solveit31_bdf created (datasets/solveit31_bdf.db, journal_mode=wal,
    schema v1) -- recording armed
In[14]:= run 1 steps 10
Out[14]= t = 1 (28 solver steps, 10 snapshots, |dE/E| = 5.616e-10)
In[15]:= sql "ATTACH DATABASE 'datasets/solveit31_adams.db' AS adams"
Out[15]= ok (0 row(s) changed)
In[16]:= sql "SELECT COUNT(*) AS paired_frames FROM states s JOIN adams.states a ON a.
    frame = s.frame AND a.body = s.body"
Out[16]= paired_frames
10
In[17]:= sql "SELECT MAX(ABS(s.y - a.y)) AS worst_method_gap FROM states s JOIN adams.
    states a ON a.frame = s.frame AND a.body = s.body"
Out[17]= worst_method_gap
5.615900633415549e-9
In[18]:= sql "DETACH DATABASE adams"
Out[18]= ok (0 row(s) changed)
In[19]:= dataset close
Out[19]= dataset solveit31_bdf closed
```

`worst_method_gap` $= 5.6 \times 10^{-9}$: two independent multistep families agreed on every recorded coordinate to solver tolerance, and the referee was a SQL `JOIN`. `DETACH` unmounts cleanly afterwards.

7.32 Differentiating a trajectory inside the database

The dataset stores both y and v_y. A backward finite difference $(y_k - y_{k-1})/(t_k - t_{k-1})$ built with `LAG(...) OVER (ORDER BY frame)` should disagree with the recorded `vy` by $g\,dt/2$ to leading order — here $9.81 \cdot 0.05/2 = 0.245250\ldots$.

Numerical analysis as a query: the window function computes the derivative estimate, the CTE wraps it, and the aggregate takes the worst error — which lands on the theoretical constant to 10

digits.

```
In[1]:= set system.g_constant = 0
In[2]:= set system.uniform_gravity = [0, -9.81, 0]
In[3]:= new sphere as ball { mass = 1, radius = 0.2, position = [0, 30, 0] }
Out[3]= obj0 as ball
In[4]:= dataset create solveit32_window
Out[4]= dataset solveit32_window created (datasets/solveit32_window.db, journal_mode=
    wal, schema v1) -- recording armed
In[5]:= run 2 steps 40
Out[5]= t = 2 (12 solver steps, 40 snapshots, |dE/E| = 1.931e-16)
In[6]:= sql "WITH fd AS (SELECT f.t AS t, s.vy AS vy, (s.y - LAG(s.y) OVER (ORDER BY s.
    frame)) / (f.t - LAG(f.t) OVER (ORDER BY s.frame)) AS v_backward FROM states s JOIN
     frames f ON f.frame = s.frame) SELECT COUNT(*) AS differences, MAX(ABS(v_backward
    - vy)) AS worst_error FROM fd WHERE v_backward IS NOT NULL"
Out[6]= differences | worst_error
39 | 0.24525000000025088
In[7]:= 9.81 * 0.05 / 2
Out[7]= 0.24525000000000002
In[8]:= dataset play
Out[8]= dataset plots written to datasets/solveit32_window_plots.html (browser
    suppressed)
In[9]:= dataset close
Out[9]= dataset solveit32_window closed
```

`worst_error = 0.24525000000025088` against the predicted `0.24525` — the leading-order truncation term measured exactly, by SQL, from a recording.

7.33 The complete dataset lifecycle in one sitting

A rock falls under Mars gravity while every `DATASET` subcommand takes a turn: `CREATE`, `SNAPSHOT`, per-`STEP` recording, `INFO`, `SAVEAS` (a checkpoint of the open dataset), more recording, `COPY`, `LIST`, `DELETE`, `OPEN`, plus a plain `VACUUM` through the SQL passthrough.

It is the grammar tour: after this transcript you have seen every production of `datasetcmd` do its job, including the checkpoint idiom (`SAVEAS` freezes 3 frames while the original grows to 4) and the refusal rules (deleting an `OPEN` dataset is refused until `CLOSE`).

```
In[1]:= set system.g_constant = 0
In[2]:= set system.uniform_gravity = [0, -3.7, 0]
In[3]:= new sphere as mars_rock { mass = 2, radius = 0.4, position = [0, 15, 0] }
Out[3]= obj0 as mars_rock
In[4]:= dataset create solveit33_tour
Out[4]= dataset solveit33_tour created (datasets/solveit33_tour.db, journal_mode=wal,
    schema v1) -- recording armed
In[5]:= dataset snapshot
Out[5]= frame 0 recorded at t = 0
In[6]:= step 0.5
Out[6]= t = 0.5 (advanced by 0.5, 12 solver steps)
In[7]:= step 0.5
Out[7]= t = 1 (advanced by 0.5, 8 solver steps)
In[8]:= dataset info
Out[8]= dataset solveit33_tour: 1 bodies, 3 frames, 3 states
```

```
In[9]:= dataset saveas solveit33_checkpoint
Out[9]= dataset solveit33_tour saved as solveit33_checkpoint
In[10]:= step 0.5
Out[10]= t = 1.5 (advanced by 0.5, 8 solver steps)
In[11]:= dataset info
Out[11]= dataset solveit33_tour: 1 bodies, 4 frames, 4 states
In[12]:= dataset info solveit33_checkpoint
Out[12]= dataset solveit33_checkpoint: 1 bodies, 3 frames, 3 states
In[13]:= sql "VACUUM"
Out[13]= ok (0 row(s) changed)
In[14]:= dataset list
Out[14]= solveit20_kepler
solveit21_free_fall
solveit22_binary
solveit23_rebounds
solveit24_exchange
solveit25_cyclotron
solveit26_dzhanibekov
solveit27_switch
solveit28_pendulum
solveit29_box_rattle
solveit30_original
solveit31_adams
solveit31_bdf
solveit32_window
solveit33_checkpoint
solveit33_tour
In[15]:= dataset close
Out[15]= dataset solveit33_tour closed
In[16]:= dataset copy solveit33_checkpoint solveit33_twin
Out[16]= dataset solveit33_checkpoint copied to solveit33_twin
In[17]:= dataset list
Out[17]= solveit20_kepler
solveit21_free_fall
solveit22_binary
solveit23_rebounds
solveit24_exchange
solveit25_cyclotron
solveit26_dzhanibekov
solveit27_switch
solveit28_pendulum
solveit29_box_rattle
solveit30_original
solveit31_adams
solveit31_bdf
solveit32_window
solveit33_checkpoint
solveit33_tour
solveit33_twin
In[18]:= dataset delete solveit33_checkpoint
Out[18]= dataset solveit33_checkpoint deleted
In[19]:= dataset delete solveit33_twin
Out[19]= dataset solveit33_twin deleted
In[20]:= dataset list
```

```
Out[20]= solveit20_kepler
solveit21_free_fall
solveit22_binary
solveit23_rebounds
solveit24_exchange
solveit25_cyclotron
solveit26_dzhanibekov
solveit27_switch
solveit28_pendulum
solveit29_box_rattle
solveit30_original
solveit31_adams
solveit31_bdf
solveit32_window
solveit33_tour
In[21]:= dataset open solveit33_tour
Out[21]= dataset solveit33_tour opened (1 bodies, 4 frames, 4 states) -- recording
    armed
In[22]:= sql "SELECT COUNT(*) AS frames FROM frames"
Out[22]= frames
4
In[23]:= dataset close
Out[23]= dataset solveit33_tour closed
```

`solveit33_checkpoint` holds 3 frames while `solveit33_tour` reaches 4 — a checkpoint is a snapshot, not a link. `LIST` tracks every create/copy/delete, and the final `OPEN` + `COUNT` proves the survivor's contents.

7.34 Two movies from one dataset

Example 19's hinged door swings under IDA while its dataset records. A movie is built *halfway* through the recording, and another at the end — from the same dataset.

Movies are derived artifacts: `movie create` just reads whatever frames exist at that moment, so one recording can wear any number of movies, and `movie delete` discards a rendering without touching a single frame of data.

```
In[1]:= set system.g_constant = 0
In[2]:= set system.uniform_gravity = [0, -9.81, 0]
In[3]:= collide off
Out[3]= collisions OFF (0 collidable pair(s); 0 impulse(s) so far)
In[4]:= new sphere as jamb { mass = 1, radius = 0.02, position = [0, 0, 0],
    inverse_mass = 0 }
Out[4]= obj0 as jamb
In[5]:= new cuboid as door { mass = 1, half_extents = [0.2, 0.4, 0.2], position =
    [0.0199986666933331, -0.9998000066665778, 0] }
Out[5]= obj1 as door
In[6]:= hinge jamb door [0, 0, 1]
Out[6]= constraint0: hinge obj0 <-> obj1 about [0, 0, 1], 5 row(s) -- one freedom left
    (METHOD IDA is required to integrate it)
In[7]:= method ida
Out[7]= method = IDA (constrained DAE, GGL index-2)
In[8]:= dataset create solveit34_door
```

```
Out[8]= dataset solveit34_door created (datasets/solveit34_door.db, journal_mode=wal,
    schema v1) -- recording armed
In[9]:= run 1 steps 50
Out[9]= t = 1 (70 solver steps, 50 snapshots, |dE/E| = 1.613e-9)
In[10]:= movie create solveit34_door as door_half
Out[10]= movie door_half created from dataset solveit34_door (movies/door_half.html, 50
     frames, 2 bodies)
In[11]:= run 1 steps 50
Out[11]= t = 2 (69 solver steps, 50 snapshots, |dE/E| = 3.782e-11)
In[12]:= movie create solveit34_door as door_full
Out[12]= movie door_full created from dataset solveit34_door (movies/door_full.html,
    100 frames, 2 bodies)
In[13]:= movie list
Out[13]= door_full
door_half
solveit20_kepler
solveit22_binary
solveit23_rebounds
solveit24_exchange
solveit25_cyclotron
solveit26_dzhanibekov
solveit28_pendulum
solveit29_box_rattle
In[14]:= sql "SELECT COUNT(*) AS frames FROM frames"
Out[14]= frames
100
In[15]:= movie delete door_half
Out[15]= movie door_half deleted
In[16]:= movie list
Out[16]= door_full
solveit20_kepler
solveit22_binary
solveit23_rebounds
solveit24_exchange
solveit25_cyclotron
solveit26_dzhanibekov
solveit28_pendulum
solveit29_box_rattle
In[17]:= movie play door_full
Out[17]= movie door_full (movies/door_full.html) ready (browser suppressed)
In[18]:= dataset close
Out[18]= dataset solveit34_door closed
```

`door_half` shows 50 frames, `door_full` 100; deleting `door_half` leaves the dataset's 100 frames untouched, and `movie play door_full` opens the swing in the browser.

7.35 A spinning torus: quaternions in the database and on the screen

A torus is symmetric about its local z axis — the *only* thing that makes its spin visible is the orientation quaternion, recorded w-first at every frame while the wheel tumbles about a tilted axis, next to an anchored marker ball.

SQL checks the recorded quaternions stay unit to roundoff (MAX(ABS($q_w^2 + q_x^2 + q_y^2 + q_z^2 - 1$)))

and that the tilted axis genuinely excites both `qx` and `qy`. The movie player rotates every ring of the torus by that quaternion — the pinned local-z drawing rule.

```
In[1]:= new torus as wheel { mass = 2, ring_radius = 1.5, tube_radius = 0.4,
    angular_velocity = [1.5, 0, 2.5] }
Out[1]= obj0 as wheel
In[2]:= new sphere as marker { mass = 1, radius = 0.1, position = [3, 0, 0], velocity =
     [0, 0.6, 0], inverse_mass = 0 }
Out[2]= obj1 as marker
In[3]:= set system.g_constant = 0
In[4]:= dataset create solveit35_torus
Out[4]= dataset solveit35_torus created (datasets/solveit35_torus.db, journal_mode=wal,
     schema v1) -- recording armed
In[5]:= run 6 steps 240
Out[5]= t = 6 (2012 solver steps, 240 snapshots, |dE/E| = 1.394e-11)
In[6]:= sql "SELECT MAX(ABS(qw*qw + qx*qx + qy*qy + qz*qz - 1.0)) AS worst_unit_error
    FROM states"
Out[6]= worst_unit_error
4.440892098500626e-16
In[7]:= sql "SELECT b.name, MAX(ABS(s.qx)) AS qx_swing, MAX(ABS(s.qy)) AS qy_swing FROM
     states s JOIN bodies b ON b.body = s.body GROUP BY b.name ORDER BY b.name"
Out[7]= name | qx_swing | qy_swing
marker | 0 | 0
wheel | 0.2891928067948092 | 0.2803142649501159
In[8]:= angmom
Out[8]= [3.675, 0, 11.850000000000001]
In[9]:= movie create solveit35_torus
Out[9]= movie solveit35_torus created from dataset solveit35_torus (movies/
    solveit35_torus.html, 240 frames, 2 bodies)
In[10]:= dataset close
Out[10]= dataset solveit35_torus closed
```

`worst_unit_error` $= 4.4 \times 10^{-16}$: the integrator's quaternion renormalization, witnessed in the recording. In the movie the tube rings visibly roll while the ring circles precess — spin you could not see on a featureless sphere.

7.36 The teacher's experiment — change a mass, rerun, subtract

Two trials of the same two-body system over the *same* space-time domain (identical start, identical `run 2 steps 40` grid); the only difference is the planet's mass, 1 in trial A and 1.5 in trial B. The mutual dynamics change, so the trajectories drift apart from $t = 0$.

`DATASET DIFF` is trajectory algebra with its dimensions checked: same frame count, bit-equal time grid, same body set — else it errors naming the first mismatch. And the algebra must *close*: rebuilding trial A as $(A - B) + B$ and diffing against the original must give zero to roundoff. SQL reads both facts out of the difference datasets.

```
In[1]:= set system.g_constant = 1
In[2]:= new sphere as sun { mass = 3, radius = 0.1, position = [-0.25, 0, 0], velocity
    = [0, -0.63245553203367588, 0] }
Out[2]= obj0 as sun
In[3]:= new sphere as planet { mass = 1, radius = 0.05, position = [0.75, 0, 0],
    velocity = [0, 1.8973665961010276, 0] }
```

```
Out[3]= obj1 as planet
In[4]:= dataset create solveit36_trial_a
Out[4]= dataset solveit36_trial_a created (datasets/solveit36_trial_a.db, journal_mode=
    wal, schema v1) -- recording armed
In[5]:= run 2 steps 40
Out[5]= t = 2 (183 solver steps, 40 snapshots, |dE/E| = 3.639e-9)
In[6]:= dataset close
Out[6]= dataset solveit36_trial_a closed
In[7]:= reset
Out[7]= system reset
In[8]:= set system.g_constant = 1
In[9]:= new sphere as sun { mass = 3, radius = 0.1, position = [-0.25, 0, 0], velocity
    = [0, -0.63245553203367588, 0] }
Out[9]= obj0 as sun
In[10]:= new sphere as planet { mass = 1.5, radius = 0.05, position = [0.75, 0, 0],
    velocity = [0, 1.8973665961010276, 0] }
Out[10]= obj1 as planet
In[11]:= dataset create solveit36_trial_b
Out[11]= dataset solveit36_trial_b created (datasets/solveit36_trial_b.db, journal_mode
    =wal, schema v1) -- recording armed
In[12]:= run 2 steps 40
Out[12]= t = 2 (199 solver steps, 40 snapshots, |dE/E| = 3.197e-9)
In[13]:= dataset close
Out[13]= dataset solveit36_trial_b closed
In[14]:= dataset diff solveit36_trial_a solveit36_trial_b as solveit36_delta
Out[14]= dataset solveit36_delta = solveit36_trial_a - solveit36_trial_b (40 frames, 2
    bodies)
In[15]:= dataset open solveit36_delta
Out[15]= dataset solveit36_delta opened (2 bodies, 40 frames, 80 states) -- recording
    armed
In[16]:= sql "SELECT MAX(ABS(x)) AS max_dx, MAX(ABS(y)) AS max_dy FROM states"
Out[16]= max_dx | max_dy
0.26778294538839353 | 0.4995465204100412
In[17]:= sql "SELECT t, MAX(ABS(x)) AS dx_at_t FROM states JOIN frames USING(frame)
    GROUP BY frame ORDER BY frame DESC LIMIT 3"
Out[17]= t | dx_at_t
2 | 0.26778294538839353
1.95 | 0.26566104547420744
1.9 | 0.26312812645046635
In[18]:= dataset close
Out[18]= dataset solveit36_delta closed
In[19]:= dataset play solveit36_delta
Out[19]= dataset plots written to datasets/solveit36_delta_plots.html (browser
    suppressed)
In[20]:= dataset sum solveit36_delta solveit36_trial_b as solveit36_rebuilt
Out[20]= dataset solveit36_rebuilt = solveit36_delta + solveit36_trial_b (40 frames, 2
    bodies)
In[21]:= dataset diff solveit36_rebuilt solveit36_trial_a as solveit36_zero
Out[21]= dataset solveit36_zero = solveit36_rebuilt - solveit36_trial_a (40 frames, 2
    bodies)
In[22]:= dataset open solveit36_zero
Out[22]= dataset solveit36_zero opened (2 bodies, 40 frames, 80 states) -- recording
    armed
In[23]:= sql "SELECT MAX(ABS(x)) AS x_residual, MAX(ABS(vy)) AS vy_residual FROM states
```

```
    "
Out[23]= x_residual | vy_residual
6.938893903907228e-18 | 2.7755575615628914e-17
In[24]:= sql "SELECT MAX(ABS(energy)) AS e_residual FROM frames"
Out[24]= e_residual
0
In[25]:= dataset close
Out[25]= dataset solveit36_zero closed
In[26]:= dataset delete solveit36_rebuilt
Out[26]= dataset solveit36_rebuilt deleted
In[27]:= dataset delete solveit36_zero
Out[27]= dataset solveit36_zero deleted
```

The difference grows to `max_dx = 0.26778294538839353` over the two seconds — a changed mass is a changed orbit — while the closure residual is `6.938893903907228e-18` in position and `2.8e-17` in velocity: pure floating-point roundoff from two passes of component arithmetic, with the energy column closing to exactly `0`. `dataset play` renders the difference curves themselves.

7.37 Cartesian to spherical, and to plane polar — with the equator as witness

A planar two-body orbit ($z = 0$ throughout, the centre of mass pinned at the origin) is recorded, then converted: `DATASET MAP ... TO spherical` and, because the trajectory is planar, `... TO polar2` — the two-dimensional chart, whose `|z| <= 1e-9` requirement is checked row by row.

A $z = 0$ orbit must sit on the sphere's equator: the mapped dataset's polar-angle column has to be $\pi/2$ at *every* one of the 100 frames — a `MIN(y) = MAX(y)` one-liner in SQL. The polar map then hands over the planet's radial extremes about the origin directly as column minima and maxima, no coordinate arithmetic in the query at all.

```
In[1]:= set system.g_constant = 1
In[2]:= new sphere as sun { mass = 3, radius = 0.1, position = [-0.25, 0, 0], velocity
    = [0, -0.63245553203367588, 0] }
Out[2]= obj0 as sun
In[3]:= new sphere as planet { mass = 1, radius = 0.05, position = [0.75, 0, 0],
    velocity = [0, 1.8973665961010276, 0] }
Out[3]= obj1 as planet
In[4]:= method adams
Out[4]= method = CVODE Adams
In[5]:= dataset create solveit37_orbit
Out[5]= dataset solveit37_orbit created (datasets/solveit37_orbit.db, journal_mode=wal,
     schema v1) -- recording armed
In[6]:= run 12.418264743622237 steps 100
Out[6]= t = 12.418264743622238 (664 solver steps, 100 snapshots, |dE/E| = 4.162e-9)
In[7]:= dataset close
Out[7]= dataset solveit37_orbit closed
In[8]:= dataset map solveit37_orbit to spherical as solveit37_sph
Out[8]= dataset solveit37_sph = spherical(solveit37_orbit) (100 frames mapped;
    velocities stored as NULL -- curvilinear velocity components need the metric)
In[9]:= dataset open solveit37_sph
Out[9]= dataset solveit37_sph opened (2 bodies, 100 frames, 200 states) -- recording
    armed
In[10]:= sql "SELECT MIN(y) AS theta_min, MAX(y) AS theta_max FROM states"
```

```
Out[10]= theta_min | theta_max
1.5707963267948966 | 1.5707963267948966
In[11]:= sql "SELECT COUNT(*) AS null_velocity_rows FROM states WHERE vx IS NULL"
Out[11]= null_velocity_rows
200
In[12]:= dataset close
Out[12]= dataset solveit37_sph closed
In[13]:= dataset map solveit37_orbit to polar2 as solveit37_pol
Out[13]= dataset solveit37_pol = polar2(solveit37_orbit) (100 frames mapped; velocities
     stored as NULL -- curvilinear velocity components need the metric)
In[14]:= dataset open solveit37_pol
Out[14]= dataset solveit37_pol opened (2 bodies, 100 frames, 200 states) -- recording
    armed
In[15]:= sql "SELECT MIN(x) AS r_min, MAX(x) AS r_max FROM states WHERE body = (SELECT
    body FROM bodies WHERE name = 'planet')"
Out[15]= r_min | r_max
0.7500000019886397 | 3.0000000158618207
In[16]:= dataset close
Out[16]= dataset solveit37_pol closed
In[17]:= dataset play solveit37_sph
Out[17]= dataset plots written to datasets/solveit37_sph_plots.html (browser suppressed
    )
In[18]:= cart_to_spherical([0, 0, 2])
Out[18]= [2, 0, 0]
In[19]:= spherical_to_cart(cart_to_spherical([1.3, 0.7, 2.1]))
Out[19]= [1.2999999999999996, 0.6999999999999997, 2.1]
In[20]:= cart2_to_polar([3, 4])
Out[20]= [5, 0.9272952180016122]
In[21]:= polar_to_cart2(cart2_to_polar([3, 4]))
Out[21]= [3, 3.9999999999999996]
```

`theta_min = theta_max = 1.5707963267948966` — $\pi/2$ to the last bit, 100 frames without exception. The planet's polar radius spans `[0.7500000019886397, 3.0000000158618207]` against the analytic $[3/4 \cdot 1,\ 3/4 \cdot 4] = [0.75, 3]$ (its share of the relative orbit's perihelion 1 and aphelion 4). All 200 velocity rows come back NULL — curvilinear velocity components need the metric, and the map refuses to fake them. The value-level calls close their own loops: `spherical_to_cart(cart_to_spherical(v))` returns `v`, and the 3-4-5 triangle reads `[5, 0.9272952180016122]` in polar form.

7.38 One helix through every remaining chart

Example 25's helical cyclotron (gyroradius 5/9, axial drift 1) is recorded once, then mapped through four genuinely three-dimensional charts — cylindrical, toroidal ($a = 2$), bispherical ($a = 2$), oblate spheroidal ($a = 1$) — and the value level round-trips every system the earlier examples have not already exercised, including the 2-D elliptic chart and the conformal 2-D bipolar chart.

Fourteen coordinate systems, fourteen inverses, all with error checking — this example is the coverage proof. Each mapped dataset is sanity-checked in SQL (the cylindrical `z` column still ends at the helix pitch `0.6981317007977318`; the toroidal `tau` column stays positive and finite), and each `X_to_cart(cart_to_X(w))` composition must return `w` to machine precision.

```
In[1]:= set system.g_constant = 0
```

```
In[2]:= set system.b_field = [0, 0, 6]
In[3]:= new sphere as ion { mass = 2, charge = -3, radius = 0.05, position = [0, 0, 0],
     velocity = [5, 0, 1] }
Out[3]= obj0 as ion
In[4]:= method bdf
Out[4]= method = CVODE BDF
In[5]:= dataset create solveit38_helix
Out[5]= dataset solveit38_helix created (datasets/solveit38_helix.db, journal_mode=wal,
     schema v1) -- recording armed
In[6]:= run 0.6981317007977318 steps 100
Out[6]= t = 0.6981317007977317 (303 solver steps, 100 snapshots, |dE/E| = 2.048e-8)
In[7]:= dataset close
Out[7]= dataset solveit38_helix closed
In[8]:= dataset map solveit38_helix to cylindrical as solveit38_cyl
Out[8]= dataset solveit38_cyl = cylindrical(solveit38_helix) (100 frames mapped;
    velocities stored as NULL -- curvilinear velocity components need the metric)
In[9]:= dataset open solveit38_cyl
Out[9]= dataset solveit38_cyl opened (1 bodies, 100 frames, 100 states) -- recording
    armed
In[10]:= sql "SELECT MAX(z) AS pitch FROM states"
Out[10]= pitch
0.6981317007977317
In[11]:= dataset close
Out[11]= dataset solveit38_cyl closed
In[12]:= dataset map solveit38_helix to toroidal 2.0 as solveit38_tor
Out[12]= dataset solveit38_tor = toroidal(solveit38_helix) (100 frames mapped;
    velocities stored as NULL -- curvilinear velocity components need the metric)
In[13]:= dataset open solveit38_tor
Out[13]= dataset solveit38_tor opened (1 bodies, 100 frames, 100 states) -- recording
    armed
In[14]:= sql "SELECT MIN(y) AS tau_min, MAX(y) AS tau_max FROM states"
Out[14]= tau_min | tau_max
5.28650864731464e-9 | 1.1892318293548387
In[15]:= dataset close
Out[15]= dataset solveit38_tor closed
In[16]:= dataset map solveit38_helix to bispherical 2.0 as solveit38_bis
Out[16]= dataset solveit38_bis = bispherical(solveit38_helix) (100 frames mapped;
    velocities stored as NULL -- curvilinear velocity components need the metric)
In[17]:= dataset map solveit38_helix to oblate 1.0 as solveit38_obl
Out[17]= dataset solveit38_obl = oblate(solveit38_helix) (100 frames mapped; velocities
     stored as NULL -- curvilinear velocity components need the metric)
In[18]:= dataset info solveit38_bis
Out[18]= dataset solveit38_bis: 1 bodies, 100 frames, 100 states
In[19]:= dataset info solveit38_obl
Out[19]= dataset solveit38_obl: 1 bodies, 100 frames, 100 states
In[20]:= let w = [1.3, 0.7, 2.1]
Out[20]= w set
In[21]:= norm(oblate_to_cart(cart_to_oblate(w, 1.25), 1.25) - w)
Out[21]= 0.00000000000000044408920985006261
In[22]:= norm(prolate_to_cart(cart_to_prolate(w, 1.25), 1.25) - w)
Out[22]= 0.00000000000000086711190182627341
In[23]:= norm(paraboloidal_to_cart(cart_to_paraboloidal(w)) - w)
Out[23]= 0.00000000000000049650683064945461
In[24]:= norm(elliptic_cyl_to_cart(cart_to_elliptic_cyl(w, 1.25), 1.25) - w)
```

```
Out[24]= 0
In[25]:= norm(parabolic_cyl_to_cart(cart_to_parabolic_cyl(w)) - w)
Out[25]= 0
In[26]:= norm(bipolar_cyl_to_cart(cart_to_bipolar_cyl(w, 1.25), 1.25) - w)
Out[26]= 0.00000000000000022204460492503131
In[27]:= norm(bispherical_to_cart(cart_to_bispherical(w, 1.25), 1.25) - w)
Out[27]= 0.00000000000000044408920985006262
In[28]:= norm(cylindrical_to_cart(cart_to_cylindrical(w)) - w)
Out[28]= 0
In[29]:= elliptic_to_cart2(cart2_to_elliptic([1.2, -0.8], 1.5), 1.5)
Out[29]= [1.2000000000000004, -0.8000000000000002]
In[30]:= bipolar_to_cart2(cart2_to_bipolar([1.2, -0.8], 1.5), 1.5)
Out[30]= [1.2, -0.8]
```

Every round trip lands between `0` exactly and `2.2e-16` — one `f64` unit in the last place — and the 2-D charts return `[1.2, -0.8]` verbatim. Singular loci stay refused: type `cart_to_toroidal([1, 0, 0], 1)` interactively and the reply names the focal ring instead of returning a NaN.

8 Watching it: the scene window, videos, dataset plots and movies

8.1 The live window

Type `SCENE CREATE` and a browser tab opens. That tab *is* the simulator's window: it draws every body on a 3-D canvas, with a toolbar (Start, Pause, Stop, Reverse, Reset, single-step, a *dt* box, zoom, grid/trails/labels toggles) and a status bar showing the playback mode, the time, *dt*, the total energy, the body count, the camera and the frame rate.

Behind it is a web server written on the Rust standard library — including the HTTP parsing, the WebSocket handshake, the SHA-1 and the base64. No dependency, and the page itself is vanilla JavaScript and canvas with no CDN fetches.

Two things about it are worth knowing. **The window evolves its own copy of the system**: typing `STEP` in the notebook does not move the window, and pressing Start in the window does not move the notebook. That isolation is what lets the window run without ever locking the language. And **Reverse is replay, not negative-time integration**: the playback keeps a ring of snapshots and walks it backward, returning to $t = 0$ bit-identically.

8.2 Recorded videos

A live window needs a live program. For something you can send to someone else, record it:

```
cargo build --release -p posim
recorder/src/record_video.py videos/scenes/kepler_ellipse.posim \
     -o videos/kepler_ellipse.html --frames 360 --dt 0.02 \
     --title "Kepler orbit, e = 0.6"
```

The result is a single HTML file: the frames embedded as data plus a canvas player. It fetches nothing, so it works from `file://` on a machine with no network, forever. The recorder drives `posim --machine` and asks it to `STEP` — **every advance is a real SUNDIALS step**. The tool is a camera, not a physics engine.

file	what to watch	measured
`kepler_ellipse.html`	the speed swinging between perihelion and aphelion	$\lvert dE\rvert/E = 9.8 \times 10^{-8}$, $\lvert dL\rvert/\lvert L\rvert = 1.3 \times 10^{-7}$
`tumbling_racket.html`	the tennis-racket flip, seen from outside	$\lvert d\lvert L\rvert\rvert/\lvert L\rvert = 0$ **exactly**; $\lvert dE\rvert/E = 6.4 \times 10^{-9}$
`box_of_shapes.html`	a cylinder, a disk and a cuboid in a rigid `BOX` 4; gold arrows are the analytic contact normals, sized by impulse	36 collisions, $\lvert dE\rvert/E = 3.4 \times 10^{-16}$
`double_pendulum_hinge.html`	two `HINGE` **joints** assembled into the classic chaotic linkage — gold rings mark the joints, gold lines their axes	the joints hold to $\lvert g\rvert = 5.6 \times 10^{-8}$ through 400 IDA steps
`universal_joint.html`	a `UNIVERSAL` **joint** carrying a driven shaft's rotation across to a second shaft, braced by a rod to a post	the bend stops at $\cos\beta = 0.6000004$ against a geometric bound of exactly 0.6; three joints hold to $\lvert g\rvert = 4.0 \times 10^{-7}$
`ball_joint_chain.html`	four links on `BALL` **joints** — the chain whirls out of the plane it started on, which a hinged chain cannot do	the four joints hold to $\lvert g\rvert = 3.3 \times 10^{-9}$; $\lvert z\rvert$ runs from exactly 0 to 1.7147
`rod_pendulum_chain.html`	four bobs on four `CONSTRAIN` **rods** — one row each, the cheapest linkage in the language, going chaotic	run continuously at the default tolerance the rods hold to $\lvert g\rvert = 5.4 \times 10^{-15}$, i.e. roundoff
`spinning_top.html`	a top on a `BALL` **joint**, precessing under gravity	1.020440 rad/s against the exact $\Omega = Mgr/(I_3\omega_3) = 1.020408$, 3 parts in 10^5
`gyroscope_gimbal.html`	a rotor in **two gimbal rings**, three perpendicular `HINGE` axes	total $L\cdot\hat{y}$ conserved to 1.4×10^{-14}; every centre stays on the pivot to 1.2×10^{-34}
`cardan_gear.html`	**Cardan gears** — a wheel in a ring of twice its radius, rolling on a `GEAR` row, its rim point tracing a straight line	line held to 1.1×10^{-8}; stroke exactly $\pm 2r$
`rack_and_pinion.html`	a weight on a `RACK` winding up a flywheel, on a `PRISMATIC` guide	falls at exactly $g/2$, and at the same rate for two different pitch radii
`piston_crankshaft.html`	the **slider-crank** — a piston, a connecting rod and a crankshaft	follows the exact kinematics to 8.4×10^{-8}; stroke exactly $L - a$ to $L + a$
`cardan_compass.html`	the same rings with a **pendulous** bowl — a ship's compass, seeking level	periods 1.878587 and 2.307339 s against measured 1.883426 and 2.313653

That last one is the one to open if you only open one, because it shows a joint doing something a rod

cannot. A `UNIVERSAL` holds a shared point and one right angle between its trunnions; it deliberately does *not* hold the two shafts straight. Left dangling, the output shaft therefore does not transmit rotation at all — it swings down past the joint like a pendulum and folds back at 176°. Bracing it with a second bearing looks right and fails at once, because `HINGE` 5 + `UNIVERSAL` 4 + `HINGE` 5 is 14 rows on 12 freedoms and three of them are redundant, which leaves the constrained solve singular. One `CONSTRAIN` — a single row — braces it instead, and pins the bend to a closed form: the shaft sweeps a cone, and $\cos\beta$ cannot go below 0.6. The recording touches 0.6000004. Grammar §12.10 works the arithmetic through.

The player has Play/Pause (or Space), frame stepping (or ←→), a scrub bar, speed from 0.25× to 4×, drag to orbit, wheel to zoom, and toggles for trails, labels and contact arrows. The corner readout shows the frame number, t, E, $|P|$, $|L|$ and the collision count *for the frame you are on* — so you can stop on the moment something looks wrong and read the conserved quantities off it.

8.3 The dataset layer: SQLite recording

Every run can record itself into a **dataset** — one ordinary SQLite database file under `datasets/<name>.db` — and every Jupyter notebook in this repository creates one. Type `dataset create myrun` before a `RUN`, and every output snapshot lands in the database: time, energy, momentum, angular momentum, centre of mass, collision count, and the full state of every body (position, w-first orientation quaternion, velocity, angular velocity). Then ask the database questions with the `sql` command, which passes *any* SQLite statement through verbatim — the sixteen worked examples 20–35 above use joins, CTEs, window functions, `ALTER TABLE`, `ATTACH` and `VACUUM` on real recordings. `dataset play` renders the recorded time series (energy, $|P|$ and $|L|$, per-body positions, a top view) into a self-contained page `datasets/<name>_plots.html` and opens it in your browser.

The simulator itself contains **no SQLite code** — its zero-external-dependency guarantee is untouched. All database work happens in the `solveit_db` sidecar (the `dataset_tools/` package, built once with `cd dataset_tools && cargo build -release`), which uses an `r2d2` connection pool shared across worker threads, switches every connection into **WAL journal mode** at startup, and sets a five-second **busy timeout** — the standard recipe that lets several readers and a writer share one SQLite file without `database is locked`. Concretely: a notebook's Python `sqlite3` cell reads the very file the simulator still holds open. The full command reference is `grammar.md` §5.14; the schema, concurrency tests and the 10-thread stress demonstration are documented in `dataset_tools/README.md`.

8.4 Movies: animations derived from datasets

A **movie** is a self-contained 3-D player page generated *from* a dataset — never a recording pipeline of its own. `movie create <dataset>` reads the `bodies`, `frames` and `states` tables and writes `movies/<name>.html`: play/pause/step, a scrub bar, 0.25×–4× speed, an orbiting camera (drag, wheel, arrow keys), per-body trails and labels, and a live readout of t, E, $|P|$, $|L|$ and the collision count. It obeys the same pinned drawing rules as the scene window and the recorded videos: wall slabs are never drawn and never steer the camera auto-fit (a `BOX` appears as the dashed wireframe), and every round shape is rendered quaternion-rotated about its local z axis so spin is visible. `movie play <name>` opens one in the browser; `movie list` and `movie delete <name>` manage them. Because the dataset is the source of truth, a movie can be rebuilt at any time without re-running any physics — Example 34 builds two different movies from one recording.

The thirteen `videos/*.html` pages of §8.2 remain: they are the legacy recorder's output, byte-reproducible and still verified. The dataset → movie pipeline is the canonical path for new work, because the movie is backed by a queryable database instead of numbers frozen into HTML.

9 How you know the numbers are right

9.1 The solver library against its C original

`sundials_rs/` ships the upstream SUNDIALS example programs, translated. Each one is run and its output **diffed byte-for-byte** against the output of the original C program. The results, the divergences, and the reasons for each divergence are in `sundials_rs/VERIFICATION.md`, `sundials_rs/differences/` and `sundials_rs/c-results/`.

Two consequences of taking this seriously: printed floating-point values go through helpers that reproduce C's `printf` conversions exactly (Rust's own `{:e}` formats exponents differently), and `pow` was made host-independent so that at least that function cannot vary between machines.

9.2 The simulator against closed-form physics

Six example programs, each printing `SUCCESS` or `FAILURE` and exiting nonzero if it fails: `kepler_orbit`, `outer_solar_system`, `tumbling_body`, `charged_in_b_field`, `newtons_cradle`, `bouncing_ball_restitution`.

9.3 The test suite

605 tests: 40 library, 19 collision, 9 conservation, 109 language, 92 quantum, 233 special-function, 11 vendored identities and 55 documentation examples that are compiled and run as written. The collision tests in particular are analytic rather than regression-style: they assert the time of impact against a closed form, not against last week's output.

9.4 The notebooks

`dynamic_notebooks/` holds 59 runnable sessions, including 34 problems from Routh's *Dynamics of a Particle* (1898) and *Dynamics of a System of Rigid Bodies* (1905). Each derives its closed-form answer in its own header and then checks the integrator against it — so the numbers in the catalogue are measured, not quoted.

9.5 And for the 7.8.0 upgrade specifically

Everything above was run against **both** the old 7.7.0 engine and the new 7.8.0 one, and the outputs diffed. They are identical byte for byte. See `PORT_7.8.0_PROVENANCE.md` and `evidence/port-7.8.0/`.

10 When something goes wrong

Errors name the column, the field, or the feature at fault, and never abort your session.

symptom	**what is actually happening**
$\lvert dE/E \rvert$ is huge and you expected conservation	Check `system.g_constant` (§3.2), the restitution of your bodies, and whether a field is doing work.
`RUN` overshoots where you meant to stop	`RUN` takes a **duration**.
bodies pass through each other	You did not `COLLIDE ON`.
`SPRK method requires a separable Hamiltonian`	A magnetic field, a magnetic moment, an external torque or a spinning body is present. The message names which. Use `METHOD ADAMS` or `BDF`.
`no object obj7` after a `DEL`	Deleting renumbers. Use `NEW ... AS` *name*.
`unknown name 'x'`	A bare identifier resolves at execution time. Bind it with `LET`, pass it as a function parameter, or use *name*`.`*field*.
a magnetised body will not spin	Multiply your moment tensor by your field by hand — you probably have a zero column.
a quantum result is off by a fraction of a percent	Refine the grid and see whether it converges. Discretisation error converges; a bug does not.
`probability accumulating near the wall`	Your wavepacket has reached the domain edge and is reflecting. Widen the grid or add `QM ABSORB`.
the scene window does not open	Set `POSIM_NO_BROWSER=1` if you are headless, or open the printed `http://127.0.0.1:<port>/` yourself.

Anything the language accepts is documented in `grammar.md` — every command, every field, every builtin, with the complete EBNF grammar and a further eighteen worked examples.

11 Where everything lives

path	what
`posim/`	the command language, the graphics window, the machine protocol
`physical_object/`	the physics: the union type, collisions, all time integration
`quantum/`	1-D, 2-D and 3-D quantum mechanics
`special_functions/`	Bessel, Legendre, Hankel, Airy, gamma, Wigner, quadrature, eigenproblems
`sundials_rs/`	the pure-Rust SUNDIALS 7.8.0 engine (vendored, read-only)
`jupyter/`	the JupyterLab kernel
`notebooks/`	one executed Jupyter notebook per example (125), each owning a dataset
`dynamic_notebooks/`	59 runnable sessions, incl. 34 Routh problems
`scripts/solveit/`	the thirty-eight examples in §7
`scripts/collisions/`	twelve documented collision scripts
`videos/`	recorded browser videos; `videos/scenes/` the scripts behind them
`datasets/`	the SQLite datasets — one per notebook and per dataset example (plus their `_plots.html` pages)
`movies/`	movies derived from datasets (self-contained 3-D player pages)
`dataset_tools/`	the `solveit_db` SQLite sidecar (rusqlite + r2d2, WAL; the one package outside the pure-Rust workspace)
`tools/`	the index builder, the verifiers, `record_video.py`
`evidence/port-7.8.0/`	the logs behind every claim in §9.5

document	for
`SolveIt.md` / `.pdf`	this one: the complete solution guide
`grammar.md` / `.pdf`	every command, the full EBNF (datasets, SQL and movies in §5.14 and §13), 18 more worked examples
`ARCHITECTURE.md`	the pinned cross-module contracts
`PORT_7.8.0_PROVENANCE.md`	what the 7.8.0 upgrade changed, and the evidence it changed nothing else
`collision_detection.md` / `.pdf`	the collision science, with 12 example scripts
`scene_info.md` / `.pdf`	the graphics window, and a survey of seven other simulators
`physical_object_simulator.md` / `.pdf`	the predecessor user guide, with 14 further examples
`special_functions.md`	the special-function library in detail
`dataset_tools/README.md`	the SQLite sidecar: schema, WAL/pool/busy-timeout architecture, CLI, stress tests
`SQLITE_DATASETS_PROVENANCE.md`	the exact commands that build, test, verify and display the dataset layer
`CLAUDE.md`	the working rules for anyone modifying this repository
`index_of_entities.html`	a browsable catalogue of every named entity in the repository

Everything above was produced by the program in this repository. Every command shown can be typed. Every number shown can be reproduced.